

General Technical Documentation

"WEATHERLAB"

Team members:

Fabian Seelig, Julian Weißböck, Andrea Roithmeier
Matthias Winklehner, David Sageder

Contents

1. Project Overview.....3
1.1 Hardware Used.....3
1.2 Software Used.....3
1.3 System Architecture.....3
1.4 Addresses at a Glance.....4

2. Installing the Operating System.....5
2.1 Choosing the Model and OS.....5
2.2 Selecting the Storage.....5
2.3 Applying Pre-Configuration Settings.....6
2.4 Writing the Image.....6
2.5 First Boot.....7

3. Network Configuration – Raspberry Pi as an Access Point.....8
3.1 Installed Services.....8
3.2 NetworkManager Configuration.....8
3.3 DNS / DHCP Configuration.....8
3.4 Website Name Resolution.....9
3.5 Wi-Fi Configuration.....9
3.6 hostapd Default Configuration.....11
3.7 Wi-Fi Startup Script (startupWIFI.sh).....11
3.8 Starting the Script Automatically.....12
3.9 Sharing Internet over Wi-Fi.....12

4. Sensors – BME680.....14
4.1 The Sensor.....14
4.2 Wiring.....14
4.3 Enabling I2C.....15
4.4 Python Library.....16
4.5 Sensor Configuration.....16

5. Status LEDs for Air Quality.....18
5.1 Purpose.....18
5.2 Wiring.....18
5.3 Required Libraries.....19
5.4 Testing the LEDs.....19
5.5 Integration into the Program.....19

6. Web Application.....20
6.1 Project Folder Structure.....20
6.2 The config.py File.....20
6.3 The app.py File.....21
6.4 Interfaces.....22
6.5 Database.....23

7. Calculated Values.....24
7.1 Air Quality.....24
7.2 Air Pressure and Trend.....24
7.3 Ventilation Recommendation.....25
7.4 Other Derived Values.....26

8. User Interface.....27
8.1 Structure.....27
8.2 The Pages.....27
8.3 Design.....28
8.4 Maps.....30
8.5 Updating the Values.....30
8.6 Extras for Continuous Operation.....31
8.7 Settings Tab.....31
8.8 Locations.....32
8.9 On-Screen Keyboard.....33

9. Web Server and Autostart.....35
9.1 Service for the Web Application.....35
9.2 Apache as a Reverse Proxy.....35
9.3 Kiosk Mode on the Touch Display.....36
9.4 Overview of the Services.....37

10. Telegram Bot.....38
10.1 Creating the Bot in Telegram.....38
10.2 Structure of telegram_bot.py.....39
10.3 Alerts and Daily Report.....40
10.4 Approving Recipients.....42
10.5 The telegram-bot Service.....43
10.6 Setup with telegram-setup.sh.....44
10.7 Testing Without Real Telegram.....44
10.8 The Extension Script.....45

11. Operation and Troubleshooting.....46
11.1 Applying Changes.....46
11.2 Useful Commands.....46
11.3 Backup.....46
11.4 Problems and Solutions.....47
11.5 Notes on Continuous Operation.....48
11.6 Conclusion.....48

12. Appendix: app.py Source Code.....49
12.1 Header and Settings.....49
12.2 Shared Data Stores.....50
12.3 Database.....50
12.4 Evaluating the History.....51
12.5 Status LEDs.....52
12.6 Sensor.....53
12.7 Querying OpenWeather.....54
12.8 Raspberry Pi System Values.....56
12.9 Interfaces and Program Start.....56
12.10 Recurring Patterns in the Code.....58

1. Project Overview

In this project, we built our own weather station using a Raspberry Pi 5. The Raspberry Pi performs two tasks at once: it provides its own Wi-Fi network and, at the same time, hosts a website that displays the measured values.

The indoor values come from a BME680 sensor connected directly to the Raspberry Pi. The outdoor values are retrieved from OpenWeather over the internet. This allows the indoor and outdoor climate to be compared directly.

In addition, a Telegram bot sends threshold alerts and weather warnings to users' phones and delivers a daily report every morning (chapter 10).

A settings tab also allows Telegram recipients, the Wi-Fi, locations and the PIN to be changed directly on the display, without having to log in via SSH (sections 8.7 to 8.9).

1.1 HARDWARE USED

COMPONENT	DESCRIPTION
Raspberry Pi 5	single-board computer running Raspberry Pi OS 64-bit
Pi Touch Display 2 (7")	shows the website permanently in full screen
BME680 (CJMCU-680)	measures temperature, humidity, air pressure and air quality
3 LEDs (green, yellow, red)	show the air quality at a glance
Breadboard and jumper wires	for wiring the sensor and LEDs
LAN cable	connects the Raspberry Pi to the internet

1.2 SOFTWARE USED

SOFTWARE	TASK
hostapd	broadcasts the station's own Wi-Fi
dnsmasq	assigns IP addresses and resolves names
Python 3 with Flask	reads the sensor and provides the data
SQLite	stores the measurement history in a file
Apache2	serves the website on port 80
Chromium	displays the website on the touch display
Telegram Bot API	sends alerts, the daily report and replies to phones

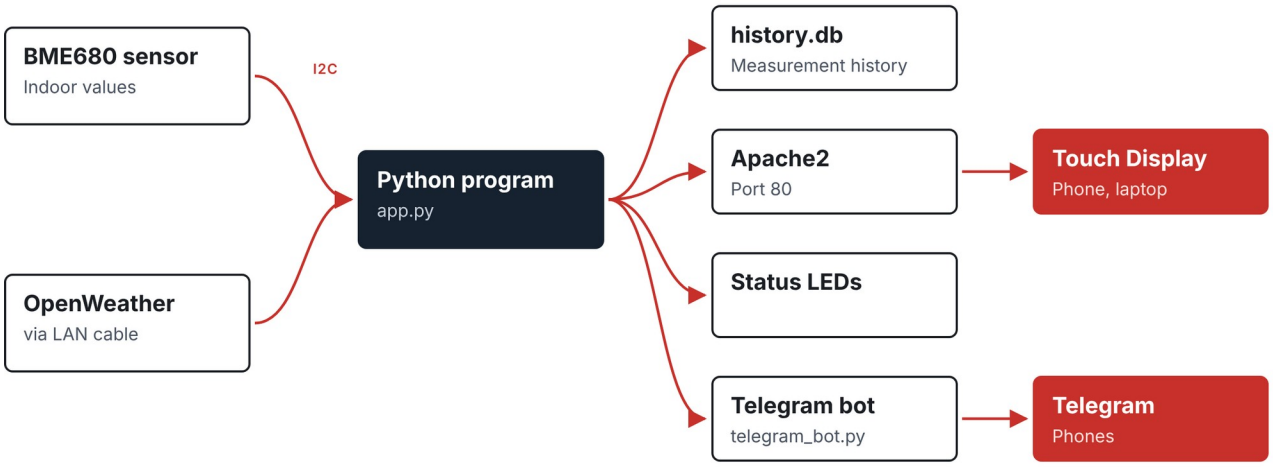
1.3 SYSTEM ARCHITECTURE

The individual components work together as follows: the sensor is connected to the Raspberry Pi via I2C and is read every second by our Python application. In the background,

the same application queries the current weather from OpenWeather every 60 seconds and stores one data point per minute in a database.

Apache receives requests from the devices and forwards them internally to our application. This makes the website available at the normal address without specifying a port.

The Telegram bot runs as a separate program alongside the web application. It retrieves the readings through the same interfaces as the website and sends messages to the recipients' phones over the internet.



1.4 ADDRESSES AT A GLANCE

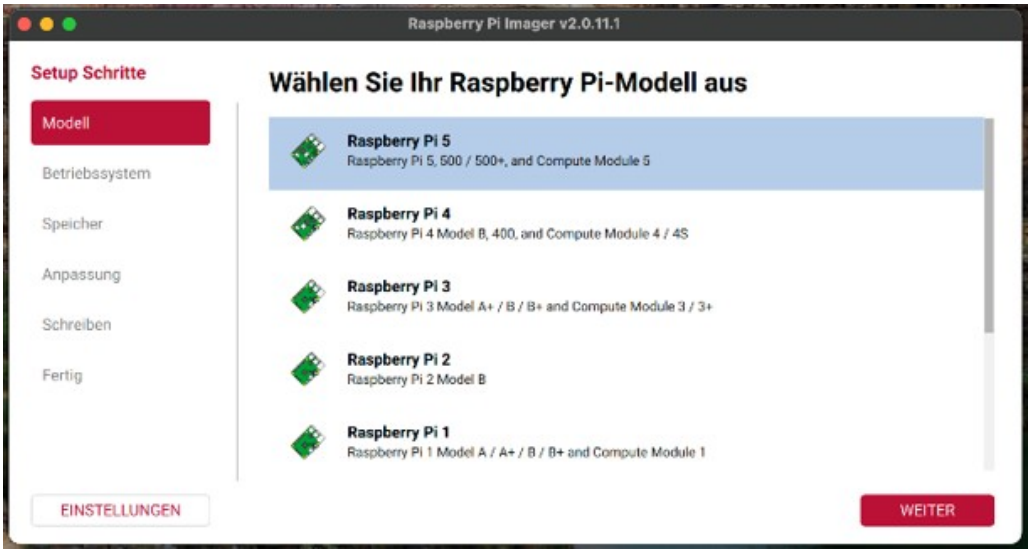
ITEM	VALUE
Hostname	weatherlab
User name	weatherlabadmin
Wi-Fi name (SSID)	WeatherLab
IP address of the Raspberry Pi (AP gateway)	192.168.10.1
Address range for devices (AP)	192.168.10.10 to 192.168.10.20
Website address	http://192.168.10.1/
Project folder	/home/weatherlabadmin/pi-dashboard

2. Installing the Operating System

The first step is to download Raspberry Pi Imager, which is available at https://www.raspberrypi.com/software/. Once the downloaded file has been installed, the SD card can be prepared.

2.1 CHOOSING THE MODEL AND OS

In the Imager, we first select the model **Raspberry Pi 5**. Next, we choose the operating system, in our case **Raspberry Pi OS (64-bit)**.





2.2 SELECTING THE STORAGE

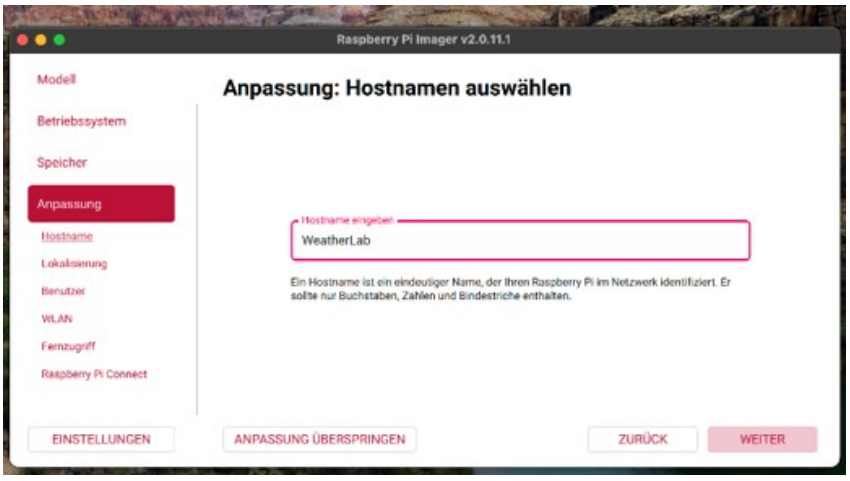
The inserted SD card is selected as the storage device. We use a 128 GB card.

2.3 APPLYING PRE-CONFIGURATION SETTINGS

The *Einstellungen* (settings) button lets you configure important options before the image is written. This saves a lot of work later, as the Raspberry Pi can then be reached over the network straight away.

SETTING	SELECTED VALUE
Hostname	weatherlab
User name	weatherlabadmin
Password	chosen freely
Capital city	Vienna (Austria)
Time zone	Europe/Vienna
Keyboard layout	at
SSH	enabled, password authentication

The SSH setting is particularly important. Without it, we would not be able to connect to the Raspberry Pi from the Windows PC later on.





2.4 WRITING THE IMAGE

After confirming, the settings are displayed and the image is written to the SD card. The process takes a few minutes, during which the storage medium must not be removed.



2.5 FIRST BOOT

The finished SD card is inserted into the Raspberry Pi, the LAN cable is connected and the device is switched on. After about a minute, you can connect via SSH:

```
ssh weatherlabadmin@weatherlab.local
```

We then bring the system up to date:

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

The first run may take a few minutes. After the reboot, reconnect via SSH.

3. Network Configuration – Raspberry Pi as an Access Point

3.1 INSTALLED SERVICES

Several services were installed and configured on the Raspberry Pi to operate the WeatherLab access point. These include:

- hostapd (broadcasts the Wi-Fi)
- dnsmasq (DHCP and DNS server)
- iptables-persistent (saves the rules for internet sharing)

They were installed with:

```
sudo apt install -y hostapd dnsmasq iptables-persistent
sudo systemctl stop hostapd
sudo systemctl stop dnsmasq
sudo systemctl unmask hostapd
```

The `unmask` command is necessary because hostapd is masked after installation and otherwise fails to start with the message *Unit hostapd.service is masked*.

The following sections explain the configuration of each service and the corresponding file paths in more detail.

3.2 NETWORKMANAGER CONFIGURATION

This configuration is located at **/etc/NetworkManager/conf.d/unmanaged.conf**.

Current versions of Raspberry Pi OS manage all network interfaces through NetworkManager. It would take the Wi-Fi interface away from hostapd, so the interface wlan0 has to be excluded.

```
[keyfile]
unmanaged-devices=interface-name:wlan0
```

Here is a line-by-line explanation of each entry:

- **[keyfile]** Section header, which must be enclosed in square brackets.
- **unmanaged-devices=interface-name:wlan0** Specifies that NetworkManager does not manage the interface wlan0. The colon before wlan0 is mandatory; with an equals sign the service no longer starts.

To check whether the setting has taken effect:

```
nmcli device status
```

For wlan0, the STATE column must show the word **unmanaged**.

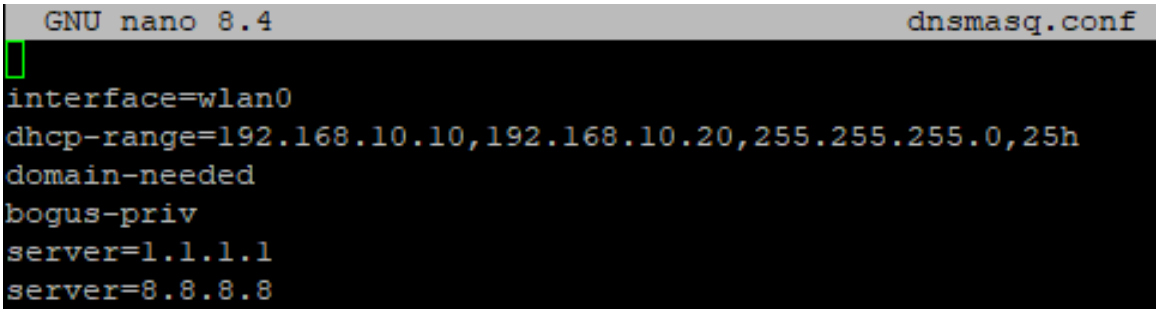
3.3 DNS / DHCP CONFIGURATION

The configuration file is located at: **/etc/dnsmasq.conf**.

```
interface=wlan0
dhcp-range=192.168.10.10,192.168.10.20,255.255.255.0,24h
domain-needed
bogus-priv
server=1.1.1.1
server=8.8.8.8
```

Here is a line-by-line explanation of each entry:

- **interface=wlan0** Specifies that dnsmasq operates on the interface wlan0. The DHCP address range is provided via this interface. The service is not active on the LAN cable.
- **dhcp-range=192.168.10.10,192.168.10.20,255.255.255.0,24h** The configured DHCP range covers 192.168.10.10 to 192.168.10.20. IP addresses from this range are assigned automatically to connected Wi-Fi clients, with a lease time of 24 hours.
- **domain-needed** Incomplete names are not forwarded to DNS servers on the internet.
- **bogus-priv** dnsmasq answers queries for private address ranges itself.
- **server=1.1.1.1 / server=8.8.8.8** These DNS servers are passed on to the clients so that names can be resolved on the Wi-Fi. It is important that only reachable addresses are entered here. An unreachable entry causes devices to connect very slowly.



3.4

WEBSITE NAME RESOLUTION

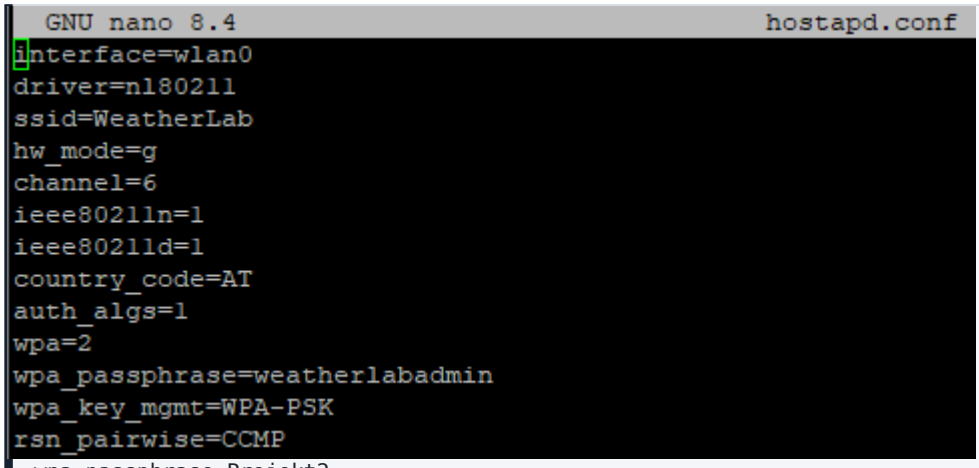
The following configuration is located at **/etc/dnsmasq.d/localips.conf**.

```
address=/weatherlab.lan/192.168.10.1
```

This file defines custom DNS resolutions. The entry ensures that the domain weatherlab.lan resolves to the IP address 192.168.10.1. This means you do not have to type the IP address in the browser but can use the name instead.

3.5 WI-FI CONFIGURATION

This is where the broadcast Wi-Fi network is configured. The configuration file is located at: **/etc/hostapd/hostapd.conf**




```
interface=wlan0
driver=nl80211
ssid=WeatherLab
hw_mode=g
channel=6
country_code=AT
auth_algs=1
wpa=2
wpa_passphrase=Projekt2.
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
```

Here is a line-by-line explanation of each entry:

- **interface=wlan0** Specifies the Wi-Fi interface used as the access point.
- **driver=nl80211** Defines the Linux Wi-Fi driver, i.e. the interface to the Wi-Fi hardware.
- **ssid=WeatherLab** Name of the Wi-Fi network (SSID) as seen by clients.
- **hw_mode=g** Sets the Wi-Fi mode to 802.11g in the 2.4 GHz band.
- **channel=6** Specifies the radio channel used (here channel 6).
- **country_code=AT** Sets the country code (Austria) for regulatory requirements.
- **auth_algs=1** Uses Open System Authentication.
- **wpa=2** Enables WPA2 encryption.
- **wpa_passphrase=Projekt2.** Password for Wi-Fi access (pre-shared key), at least 8 characters.
- **wpa_key_mgmt=WPA-PSK** Uses WPA with a pre-shared key for authentication.
- **rsn_pairwise=CCMP** Sets AES-based CCMP encryption for WPA2.

During setup, the file initially also contained the entries `ieee80211n=1` and `ieee80211d=1`. However, these lines were copied incompletely when typing them in, which caused hostapd to stop starting with the message *unknown configuration item*. As these are only optional settings for higher data rates, we left them out.

The best way to find errors in this file is:

```
sudo hostapd -dd /etc/hostapd/hostapd.conf
```

The output shows the faulty line together with its line number. Stop the test with Ctrl+C.

3.6 HOSTAPD DEFAULT CONFIGURATION

The file **/etc/default/hostapd** contains the default configuration for the hostapd service, which runs the system as a Wi-Fi access point.

Important parameters:

- **DAEMON_CONF="/etc/hostapd/hostapd.conf"** Specifies the path to the main hostapd configuration file, which is loaded at startup.

This setting ensures that the Wi-Fi access point is started automatically with the defined configuration when the system boots.

3.7 WI-FI STARTUP SCRIPT (STARTUPWIFI.SH)

In addition, a script called **startupWIFI.sh** was created. It is located in the directory **/scripts/startupWIFI.sh**.

The script starts the network services and settings required for the Wi-Fi access point. It became necessary because current versions of Raspberry Pi OS no longer use the previously common file /etc/dhcpcd.conf, so the static IP address has to be set in a different way.

```
#!/bin/bash
ip addr add 192.168.10.1/24 dev wlan0
rfkill unblock wifi
ip link set wlan0 up
systemctl restart hostapd
systemctl restart dnsmasq
```

Here is a line-by-line explanation of each entry:

- **#!/bin/bash** Specifies which program is used to run the file. If this line is missing, the script aborts with the message *Exec format error*.
- **ip addr add 192.168.10.1/24 dev wlan0** Assigns the static IP address 192.168.10.1 with the subnet /24 (255.255.255.0) to the Wi-Fi interface wlan0. This address serves as the gateway for the Wi-Fi network.
- **rfkill unblock wifi** Removes any software or hardware block on the Wi-Fi module.
- **ip link set wlan0 up** Enables the network interface wlan0.
- **systemctl restart hostapd** Restarts the hostapd service so that the Wi-Fi is broadcast.
- **systemctl restart dnsmasq** Restarts the dnsmasq service, which provides DHCP and DNS for the Wi-Fi.

The script must be made executable:

```
sudo chmod +x /scripts/startupWIFI.sh
```

When running it manually, `sudo` is required because all commands need system privileges. Without sudo, the message *RTNETLINK answers: Operation not permitted* appears.

```
 GNU nano 8.4                                    startupWIFI.sh
#!/bin/bash
ip addr add 192.168.10.1/24 dev wlan0
rfkill unblock wifi
ip link set wlan0 up
systemctl restart hostapd
systemctl restart dnsmasq
```

3.8 STARTING THE SCRIPT AUTOMATICALLY

To make sure the script runs automatically after every reboot, we created a dedicated service for it. The file is located at **/etc/systemd/system/startup-wifi.service**.

```
[Unit]
Description=WLAN Access Point Setup
After=network.target

[Service]
Type=oneshot
ExecStart=/scripts/startupWIFI.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Here is a line-by-line explanation of each entry:

- **After=network.target** The service only starts once the network is basically ready.
- **Type=oneshot** The script runs once and then exits.
- **RemainAfterExit=yes** The service is still considered active afterwards. The status *active (exited)* is therefore the correct state here.
- **WantedBy=multi-user.target** The service is started as part of the normal system boot.

It was enabled with:

```
sudo systemctl daemon-reload
sudo systemctl enable startup-wifi.service
sudo systemctl enable hostapd
sudo systemctl enable dnsmasq
```

3.9 SHARING INTERNET OVER WI-FI

The Wi-Fi on its own does not provide internet access. The Raspberry Pi gets it via the LAN cable and passes it on to the connected devices. Two things are needed for this.

First, packet forwarding must be allowed. For this purpose, we created the file **/etc/sysctl.d/99-wetterstation.conf**:

```
net.ipv4.ip_forward=1
```

We deliberately put this value in a separate file. Previously, we had only set it with `sysctl -w`, which meant it was lost after every reboot.

Then three rules were added to translate the traffic between wlan0 and eth0:

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
sudo iptables -A FORWARD -i eth0 -o wlan0 -m state --state RELATED,ESTABLISHED -j ACCEPT
sudo iptables -A FORWARD -i wlan0 -o eth0 -j ACCEPT
sudo netfilter-persistent save
sudo systemctl enable netfilter-persistent
```

Meaning of the rules:

- **POSTROUTING -o eth0 -j MASQUERADE** Packets from the Wi-Fi appear to the outside world with the address of the Raspberry Pi.
- **FORWARD -i eth0 -o wlan0 with RELATED,ESTABLISHED** Replies from the internet are allowed back into the Wi-Fi.
- **FORWARD -i wlan0 -o eth0** Requests from the Wi-Fi are allowed out to the internet.
- **netfilter-persistent save** Saves the rules permanently so that they are reloaded at boot.

To check that everything is active:

```
cat /proc/sys/net/ipv4/ip_forward
sudo iptables -t nat -L POSTROUTING -n -v
```

The first command must output the value 1.

4. Sensors – BME680

4.1 THE SENSOR

We use a BME680 on a CJMCU-680 breakout board. The sensor measures four quantities at the same time: temperature, humidity, air pressure and a gas resistance from which the air quality can be derived.

The connection to the Raspberry Pi uses the I2C bus. Only four wires are needed; the two remaining pins on the board (SDO and CS) stay unconnected, as they are only required for SPI mode.

4.2 WIRING

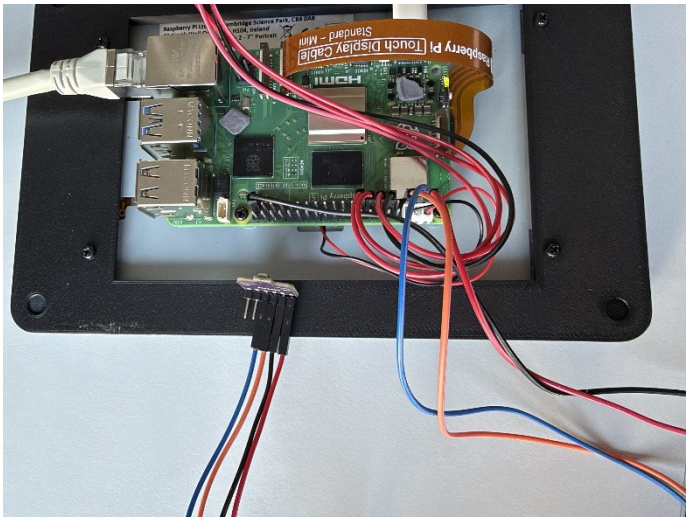
Before connecting anything, the Raspberry Pi must be shut down and disconnected from the power supply.

BME680	RASPBERRY PI	EXPLANATION
VCC	pin 1 (3.3 V)	power supply
GND	pin 39 (GND)	ground
SCL	pin 5 (GPIO 3)	clock line
SDA	pin 3 (GPIO 2)	data line

It is important that VCC is connected to 3.3 V and not to the 5 V on pin 2 or pin 4. These boards usually have no voltage regulator, and the sensor itself tolerates a maximum of 3.6 V.

Also note the order of the pins on the board: **SCL comes before SDA**, i.e. the reverse of the usual layout. If the two are swapped, the sensor will not be detected later.

In our setup, pins 2, 4 and 6 are already used by the touch display, so we used pin 39 for ground. All GND pins are connected internally, so it does not matter which one is used.



4.3 ENABLING I2C

The I2C bus is disabled by default and must be enabled first:

```
sudo raspi-config
```

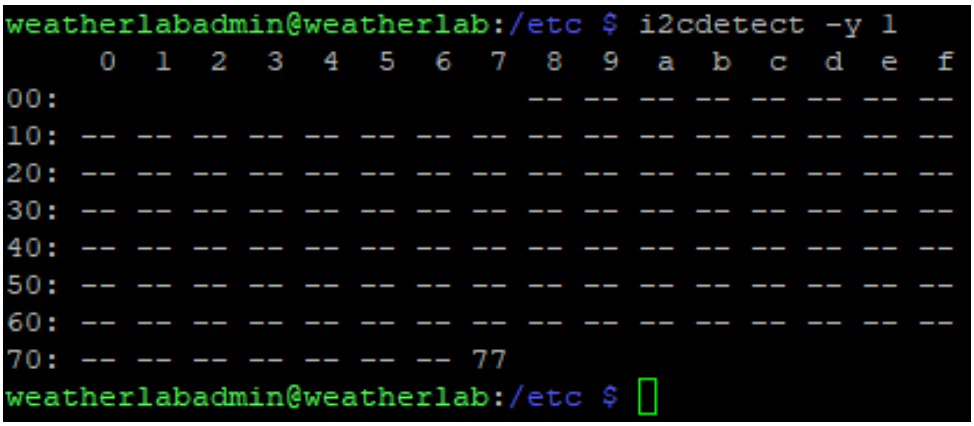
There, select *Interface Options*, then *I2C* and confirm with *Yes*. A reboot is then required:

```
sudo reboot
```

To check whether the sensor is detected:

```
sudo apt install -y i2c-tools
i2cdetect -y 1
```

The address **76** or **77** must appear in the table that is printed. On our board, the sensor responds at **0x77** because the SDO pin is tied to 3.3 V internally. If nothing appears at all, SDA and SCL are usually swapped.



To allow our application to access the bus later, the user was added to the i2c group:

```
sudo usermod -aG i2c weatherlabadmin
```

This change only takes effect after a reboot.

4.4 PYTHON LIBRARY

To read the sensor, we use the bme680 library, version 2.0.0, installed in a dedicated Python environment:

```
cd ~/pi-dashboard
python3 -m venv venv
venv/bin/pip install flask requests bme680 smbus2 psutil gpiozero
```

We ran into a problem here during setup: our program initially used the constant ENABLE_GAS_MEASUREMENT, which is called **ENABLE_GAS_MEAS** in this version. As a result, initialisation aborted with the message *module 'bme680' has no attribute 'ENABLE_GAS_MEASUREMENT'* and no values were displayed at all.

We solved this by accepting both spellings:

```
ENABLE_GAS = getattr(bme680, "ENABLE_GAS_MEASUREMENT", getattr(bme680, "ENABLE_GAS_MEAS", 1))
```

4.5 SENSOR CONFIGURATION

The sensor is configured in the program as follows:

```
sensor.set_humidity_oversample(bme680.OS_2X)
sensor.set_pressure_oversample(bme680.OS_4X)
sensor.set_temperature_oversample(bme680.OS_8X)
sensor.set_filter(bme680.FILTER_SIZE_3)
sensor.set_gas_status(ENABLE_GAS)
sensor.set_gas_heater_temperature(320)
sensor.set_gas_heater_duration(150)
sensor.select_gas_heater_profile(0)
```

Meaning of the settings:

- **Oversampling values** Specify how often the sensor measures and averages internally. Higher values mean more accurate but slower measurements.
- **set_filter(FILTER_SIZE_3)** Smooths short-term fluctuations in air pressure and temperature.
- **set_gas_heater_temperature(320)** Heats the gas measuring cell to 320 °C. Only in this heated state does it provide usable values.
- **set_gas_heater_duration(150)** Sets the heating time in milliseconds.

Because the sensor heats up to measure gases, it warms itself when measuring frequently. As a result, the displayed temperature may be about 0.5 to 1 degree too high. If required, the

SENSOR_INTERVAL setting can be used to increase the measurement interval from one to three seconds.

In addition, the gas sensor needs a burn-in period of about 24 to 48 hours after it is first switched on before its values are stable.

5. Status LEDs for Air Quality

5.1 PURPOSE

To make the air quality visible from a distance without looking at the screen, we added three LEDs. They show the same value as the website.

AIR QUALITY ON THE WEBSITE	LED
Sehr gut, Gut (very good, good)	green
Mittel (moderate)	yellow
Schlecht, Sehr schlecht (poor, very poor)	red

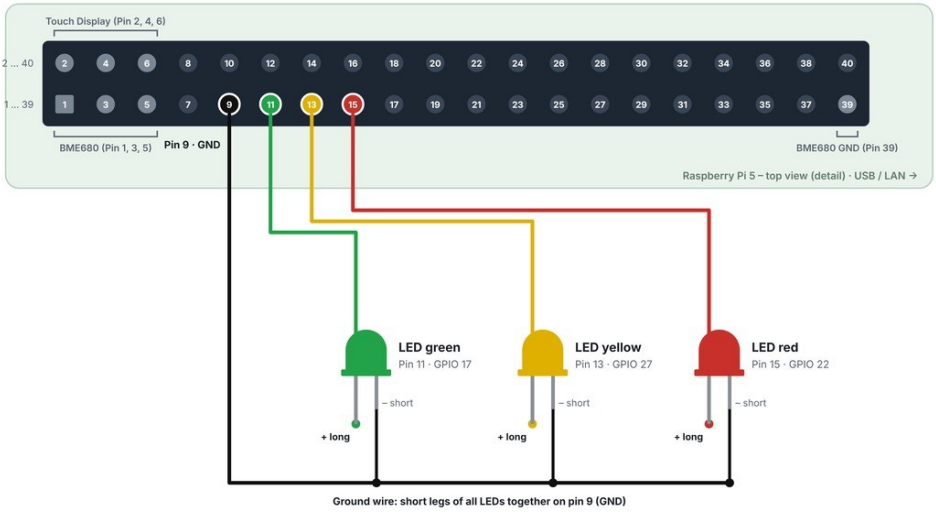
Exactly one LED is always lit; the other two are switched off when the level changes. As long as no reading is available, all three remain off.

5.2 WIRING

The LEDs are connected directly to the GPIO pins without series resistors. The long leg (positive) goes to the GPIO pin, the short leg (negative) to the common ground.

LED	LONG LEG (+) TO	SHORT LEG TO
green	pin 11 (GPIO 17)	common ground (pin 9)
yellow	pin 13 (GPIO 27)	common ground (pin 9)
red	pin 15 (GPIO 22)	common ground (pin 9)

We used pin 9 as the common ground. All three short legs can be connected to the same GND pin, as all ground pins are connected internally. On the breadboard, a single wire runs from pin 9 to the negative rail at the edge.



5.3 REQUIRED LIBRARIES

The LEDs are controlled with the gpiozero library. On the Raspberry Pi 5, this additionally requires lgpio.

```
sudo apt install -y python3-lgpio
cd ~/pi-dashboard
echo "include-system-site-packages = true" >> venv/pyvenv.cfg
venv/bin/pip install gpiozero
```

Our attempt to install lgpio directly via pip failed because the package is built from source, which requires the swig program. We therefore use the ready-made system package and allow the Python environment to use system packages as well.

5.4 TESTING THE LEDS

Before integrating them into the program, we tested the wiring separately:

```
cd ~/pi-dashboard
venv/bin/python3 -c "
from gpiozero import LED
from time import sleep
for pin, name in ((17,'grün'), (27,'gelb'), (22,'rot')):
    l = LED(pin); l.on(); print(name, 'an'); sleep(1.5); l.off(); l.close()"
```

Each LED lights up for one and a half seconds in turn. If one does not light up, the polarity is usually reversed and the LED needs to be turned round.

5.5 INTEGRATION INTO THE PROGRAM

In the program, the LEDs are set up at startup and updated with every measurement:

```
def update_leds(quality):
    if not leds:
        return
    farbe = {"Sehr gut": "gruen", "Gut": "gruen", "Mittel": "gelb",
             "Schlecht": "rot", "Sehr schlecht": "rot"}.get(quality)
    for name, led in leds.items():
        led.on() if name == farbe else led.off()
```

The mapping uses the same result that is delivered to the website, so the display and the LEDs can never get out of sync.

The pins can be changed in the configuration file:

```
LED_ENABLED = True
LED_PIN_GRUEN = 17
LED_PIN_GELB = 27
LED_PIN_ROT = 22
```

If gpiozero is not available or an error occurs when accessing it, the program continues without LEDs. The weather station therefore still works when the LEDs are disconnected.

6. Web Application

6.1 PROJECT FOLDER STRUCTURE

The entire project is contained in a single folder. Backing up this folder means you have everything in one place.

```
/home/weatherlabadmin/pi-dashboard/
├── app.py              program: sensor, weather, database, interfaces
├── config.py           settings, API key, start location, bot token, PIN
├── settings.py         settings tab: PIN check, Telegram, Wi-Fi
├── chat_store.py       list of Telegram recipients (for bot and tab)
├── locations.py        locations, place search, PIN change, renaming
├── history.db          measurement history, created automatically
├── locations.json      saved locations and active location
├── telegram_bot.py     Telegram bot (separate service)
├── telegram_chats.json approved recipients and pending requests
├── telegram_state.json active alerts, last daily report
├── weatherlab-erweiterung.sh  settings tab and new bot
├── weatherlab-standorte.sh    locations, PIN change, renaming
├── backups/            automatic backups made by both scripts
├── venv/               Python environment with the installed packages
├── templates/
│   └── index.html      structure of the website
└── static/
    ├── style.css       look of the website
    ├── app.js          logic in the browser
    ├── settings.js     settings tab and on-screen keyboard
    └── standorte.js    locations, PIN change, location picker
```

6.2 THE CONFIG.PY FILE

This file contains all values that you may want to adjust, so nothing needs to be changed in the program code itself.

```
OPENWEATHER_API_KEY = "..."
LAT = 48.3069
LON = 14.2858

SENSOR_INTERVAL = 1
WEATHER_INTERVAL = 60
FORECAST_INTERVAL = 1800

SEA_LEVEL_PRESSURE_HPA = 1013.25

GAS_QUALITY_THRESHOLDS = {
    "sehr_gut": 130000,
    "gut": 90000,
    "mittel": 50000,
    "schlecht": 20000,
}

HISTORY_INTERVAL = 60
HISTORY_DAYS = 7
NIGHT_START = 22
NIGHT_END = 6
IDLE_RETURN_SECONDS = 90

LED_ENABLED = True
LED_PIN_GRUEN = 17
LED_PIN_GELB = 27
```

```
LED_PIN_ROT = 22
SETTINGS_PIN = "......"
```

Here is a line-by-line explanation of each entry:

- **OPENWEATHER_API_KEY** The free access key from OpenWeather. It consists of 32 characters. A newly created key is rejected for up to two hours until it has been activated.
- **LAT / LON** The coordinates of the first location, in our case Linz. Since the locations extension, they are only the initial value: on first start, locations.py copies them into the file `locations.json`, after which the location is chosen in the settings (section 8.8).
- **SENSOR_INTERVAL** Interval between two BME680 measurements in seconds.
- **WEATHER_INTERVAL** Interval between OpenWeather queries. 60 seconds results in about 1,440 queries per day, which is within the free quota.
- **FORECAST_INTERVAL** Interval between queries for the 5-day forecast, here every 30 minutes.
- **SEA_LEVEL_PRESSURE_HPA** Reference pressure at sea level, required for the altitude calculation.
- **GAS_QUALITY_THRESHOLDS** Thresholds at which the air quality is assigned to each level.
- **HISTORY_INTERVAL / HISTORY_DAYS** Define how often a data point is stored and how long it is kept.
- **NIGHT_START / NIGHT_END** Period during which the display is dimmed.
- **IDLE_RETURN_SECONDS** Time without a touch after which the display returns to the home page.
- **LED_ENABLED and the LED pins** Enable the status LEDs and define the GPIO numbers used.
- **SETTINGS_PIN** The PIN for the settings tab, 4 to 8 digits. It is entered by the extension script and can be changed in the tab (section 8.7). This documentation therefore only shows a placeholder.

6.3 THE APP.PY FILE

The program is written in Python and uses Flask as its web server. At startup, four loops are launched as separate threads that run permanently in the background:

```
if __name__ == "__main__":
    init_db()
    init_leds()
    threading.Thread(target=sensor_loop, daemon=True).start()
    threading.Thread(target=weather_loop, daemon=True).start()
    threading.Thread(target=forecast_loop, daemon=True).start()
    threading.Thread(target=history_loop, daemon=True).start()
    app.run(host="0.0.0.0", port=5000, threaded=True)
```

Tasks of the individual functions:

FUNCTION	TASK
init_db()	creates the table in the database if it does not exist

FUNCTION	TASK
init_leds()	registers the three status LEDs
sensor_loop()	reads the BME680 every second
weather_loop()	fetches the current weather every 60 seconds
forecast_loop()	fetches the 5-day forecast every 30 minutes
history_loop()	stores one data point in the database every minute
get_pi_stats()	reads CPU temperature, load, RAM and storage

The loops collect the data in the background, while the interfaces only return what is already available. As a result, the website responds immediately, even if OpenWeather is slow at the moment. To prevent data being written and read at the same time, the shared data is protected by locks.

If the program does not find the sensor at startup, it tries again every 10 seconds instead of exiting. This means the website keeps running even without a sensor connected.

Since the extensions, app.py also includes two additional files: settings.py for the settings tab and locations.py for the locations (sections 8.7 and 8.8). Since then, weather_loop and forecast_loop read the coordinates from locations.py instead of config.py on every run.

6.4 INTERFACES

The website retrieves its values from several endpoints that return plain data in JSON format. They can also be called from the terminal, which is useful for troubleshooting.

ENDPOINT	RETURNS
/api/indoor	indoor values, trend, daily min/max, pressure trend
/api/outdoor	outdoor weather, air pressure, sunrise and sunset
/api/forecast	5-day forecast
/api/history	measurement history for 6 hours, 24 hours or 7 days
/api/pi-stats	CPU, RAM, storage and uptime of the Raspberry Pi
/api/location	active location with name, and settings for night mode
/api/locations	saved locations and the active location; via POST with PIN: search, add, select and remove
/api/locations/select	switch location without PIN, for the location picker on the home page
/api/settings	settings tab, POST only with PIN (Telegram and Wi-Fi)
/api/settings/pending	number of pending Telegram requests, without PIN (orange dot)
/api/settings/pin	change the PIN, using the current PIN
/api/settings/ rename_chat	rename a Telegram recipient, with PIN

The endpoints under /api/settings and /api/locations come from settings.py and locations.py. They are registered in app.py as so-called blueprints, i.e. groups of endpoints defined in a separate file.

Example call:

```
curl -s http://localhost/api/indoor
```

6.5 DATABASE

We use SQLite for the measurement history. It is a file-based database that does not need its own server process and is already included in Python. The entire database is stored in the file **history.db** in the project folder.

```
CREATE TABLE readings (
    ts       INTEGER PRIMARY KEY,   -- time stamp
    in_temp  REAL,                  -- indoor temperature
    in_hum   REAL,                  -- indoor humidity
    out_temp REAL,                  -- outdoor temperature
    out_hum  REAL,                  -- outdoor humidity
    pressure REAL                   -- air pressure
);
```

One row is written per minute, which adds up to 1,440 rows per day. The program automatically deletes entries older than eight days. With seven days of history, this amounts to around 10,000 rows and about one to two megabytes.

For the chart, the per-minute values are aggregated into larger time intervals in SQL using AVG and GROUP BY, so that 10,000 points do not have to be drawn for seven days.

The contents can be viewed if required:

```
sudo apt install -y sqlite3
sqlite3 -header -column ~/pi-dashboard/history.db "SELECT datetime(ts,'unixepoch','localtime'),
in_temp, out_temp FROM readings ORDER BY ts DESC LIMIT 5;"
```

7. Calculated Values

Not everything shown on the website is measured directly. The program calculates some values from the measured data.

7.1 AIR QUALITY

The BME680 does not provide a ready-made air quality value but an electrical resistance in ohms. A high resistance means clean air, a low one means a high level of gases such as emissions from materials, cooking fumes or stale breath. From this, we derive five levels:

LEVEL	GAS RESISTANCE
Sehr gut (very good)	from 130,000 Ω
Gut (good)	from 90,000 Ω
Mittel (moderate)	from 50,000 Ω
Schlecht (poor)	from 20,000 Ω
Sehr schlecht (very poor)	below that

```
def gas_quality_label(gas_resistance):
    t = config.GAS_QUALITY_THRESHOLDS
    if gas_resistance >= t["sehr_gut"]:
        return "Sehr gut"
    if gas_resistance >= t["gut"]:
        return "Gut"
    if gas_resistance >= t["mittel"]:
        return "Mittel"
    if gas_resistance >= t["schlecht"]:
        return "Schlecht"
    return "Sehr schlecht"
```

It is important to know that these values are relative and not calibrated. The sensor does not tell you how much CO_2 is in the air, only whether the air is better or worse than usual in this room. The thresholds should be adjusted to the room after the burn-in period.

In addition, a value is only accepted once the sensor reports `heat_stable`, i.e. the heater has reached its operating temperature. This takes one to two minutes after every start.

7.2 AIR PRESSURE AND TREND

Initially, we calculated the altitude above sea level from the air pressure:

```
def compute_altitude(pressure_hpa):
    return round(44330 * (1 - (pressure_hpa / config.SEA_LEVEL_PRESSURE_HPA) ** (1 / 5.255)), 1)
```

In practice, however, this proved impractical. Due to the weather, the air pressure changes by several hectopascals every day, so the displayed altitude kept fluctuating by a few dozen metres even though the device was not moved at all. Adjusting the reference pressure is only ever correct on the day it is set.

We therefore decided to display the air pressure itself, together with its trend over the last three hours. This is the actual weather information; old barometers with labels such as *Fair*, *Change* and *Rain* meant exactly the same thing.

CHANGE IN 3 HOURS	DISPLAY
from +3 hPa	stark steigend (rising sharply)
from +1 hPa	steigend (rising)
between −1 and +1 hPa	gleichbleibend (steady)
from −1 hPa	fallend (falling)
from −3 hPa	stark fallend (falling sharply)

Falling air pressure indicates deteriorating weather, rising pressure an improvement. For the calculation, the current value is compared with the average from the database from about three hours earlier.

7.3 VENTILATION RECOMMENDATION

From the indoor and outdoor values, the website derives a note on whether ventilating is worthwhile at the moment.

CONDITION	DISPLAY
air quality Schlecht or Sehr schlecht	Bitte lüften, die Luftqualität ist schlecht (please ventilate, air quality is poor)
humidity above 60 % and drier air outside	Lüften empfohlen, draußen ist die Luft trockener (ventilation recommended, the air outside is drier)
humidity above 60 %, outside not drier	Luft feucht, Lüften bringt gerade wenig (humid air, ventilating won't help much right now)
humidity below 35 %	Luft sehr trocken (air very dry)
temperature above 26 °C	Raum sehr warm (room very warm)
temperature below 18 °C	Raum kühl (room cool)
otherwise	Raumklima angenehm (comfortable indoor climate)

The comparison is not based on the percentages but on the absolute humidity in grams per cubic metre. This is important because cold outdoor air at 80 % can actually contain less water than warm indoor air at 60 %. It is calculated using the Magnus formula:

```
function absoluteHumidity(tempC, relHum) {
  const sat = 6.112 * Math.exp((17.67 * tempC) / (tempC + 243.5));
  return (sat * relHum * 2.1674) / (273.15 + tempC);
}
```

7.4 OTHER DERIVED VALUES

VALUE	CALCULATION
Daily min/max	lowest and highest indoor temperature since midnight, queried from the database via SQL
Temperature trend	comparison of the current value with the average from 30 minutes ago, threshold 0.3 °C
Feels-like temperature	supplied directly by OpenWeather
Wind speed	OpenWeather supplies metres per second, which we convert to km/h with a factor of 3.6

8. User Interface

8.1 STRUCTURE

The website consists of five files that work together:

FILE	TASK
templates/index.html	defines which elements exist and where they are placed
static/style.css	defines the appearance, i.e. colours, sizes and spacing
static/app.js	fetches the values from the application and fills them into the elements
static/settings.js	settings tab with PIN lock and on-screen keyboard (sections 8.7 and 8.9)
static/standorte.js	locations, PIN change, renaming and location picker on the home page (section 8.8)

Every field in the HTML has a unique identifier, for example `id="in-temp"` for the indoor temperature. app.js uses exactly these names to find where each value belongs. The lines containing `{{ url_for(...) }}` are Flask placeholders, which are replaced with the correct paths to style.css and app.js when the page is served.

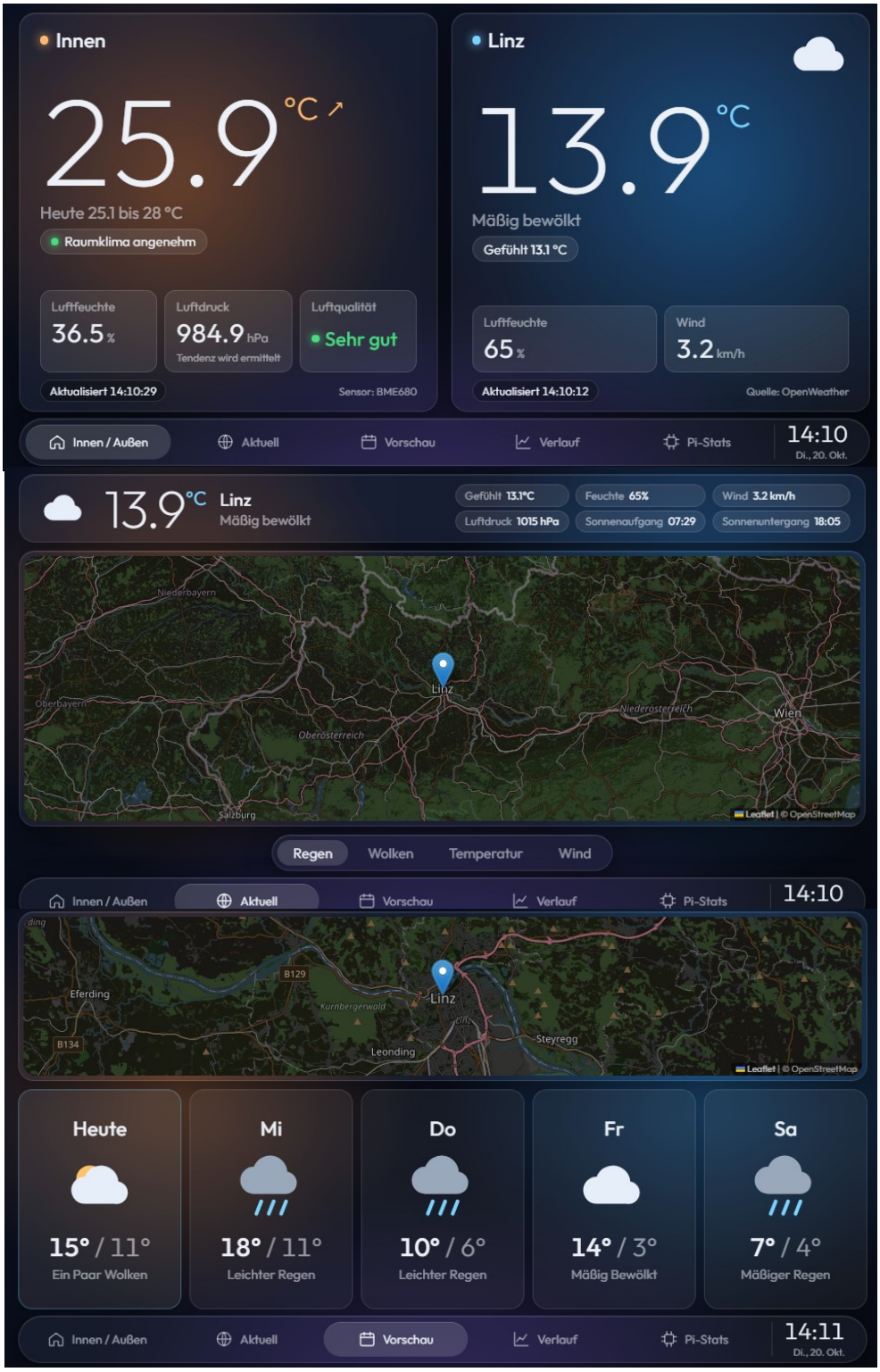
8.2 THE PAGES

The navigation bar at the bottom lets you switch between five pages. All pages are contained in the HTML, and only one of them is shown at a time, so nothing has to be reloaded when switching.

PAGE	CONTENTS
Innen / Außen (Indoor / Outdoor)	indoor temperature with trend and daily min/max, humidity, air pressure with trend, air quality, ventilation recommendation and, next to it, the current outdoor weather; tapping the location name opens the location picker
Aktuell (Current)	detailed weather with feels-like temperature, air pressure, sunrise and sunset, plus a map on which rain, clouds, temperature or wind can be overlaid
Vorschau (Forecast)	location map and the forecast for the next five days
Verlauf (History)	chart of indoor and outdoor temperature and humidity, switchable between 6 hours, 24 hours and 7 days
Pi-Stats	CPU temperature, CPU load, memory, storage and uptime of the Raspberry Pi

In addition, a clock with the date is shown on the right of the navigation bar and is visible on every page.

To the left of the clock is the gear icon for the settings tab (section 8.7). It is also contained in the HTML as a sixth page, but its contents only appear after the PIN has been entered.


Innen
25.9 °C ↗
Heute 25.1 bis 28 °C
Raumklima angenehm
Luftfeuchte
36.5 %
Luftdruck
984.9 hPa
Tendenz wird ermittelt
Luftqualität
Sehr gut
Aktualisiert 14:10:29
Sensor: BME680
Linz
13.9 °C
Mäßig bewölkt
Gefühlt 13.1 °C
Luftfeuchte
65 %
Wind
3.2 km/h
Aktualisiert 14:10:12
Quelle: OpenWeather
Innen / Außen
Aktuell
Vorschau
Verlauf
Pi-Stats
14:10
Di., 20. Okt.
13.9 °C
Linz
Mäßig bewölkt
Gefühlt 13.1°C
Feuchte 65%
Wind 3.2 km/h
Luftdruck 1015 hPa
Sonnenaufgang 07:29
Sonnenuntergang 18:05
Niederbayern
Oberbayern
Linz
Oberösterreich
Niederösterreich
Wien
Salzburg
Leaflet | © OpenStreetMap
Regen
Wolken
Temperatur
Wind
Innen / Außen
Aktuell
Vorschau
Verlauf
Pi-Stats
14:10
Eferding
B129
Kürnbergerwald
Linz
B134
Leonding
Steyregg
Leaflet | © OpenStreetMap
Heute
15° / 11°
Ein Paar Wolken
Mi
18° / 11°
Leichter Regen
Do
10° / 6°
Leichter Regen
Fr
14° / 3°
Mäßig Bewölkt
Sa
7° / 4°
Mäßiger Regen
Innen / Außen
Aktuell
Vorschau
Verlauf
Pi-Stats
14:11
Di., 20. Okt.



8.3 DESIGN

We chose a dark design because the display is permanently switched on and a bright background would be distracting in the room. The tiles are semi-transparent and sit over coloured light spots in the background, warm on the indoor side and cool on the outdoor side.

All sizes are specified in `vw` and `vh` units, i.e. as a percentage of the screen width and height. This means the layout adapts automatically, whether the page is opened on the 7-inch display or on a phone.

We drew the weather icons ourselves as SVG instead of loading the images from OpenWeather. This way they match the design and also work when there is no internet connection.

8.4 MAPS

For the maps, we use the Leaflet library together with tiles from OpenStreetMap. The weather layers for rain, clouds, temperature and wind come from OpenWeather.

Initially, we used dark tiles from CARTO. However, this provider now requires its own key, which is why the text *API KEY REQUIRED* was permanently displayed over the map. We therefore switched to the standard OpenStreetMap tiles, which do not need a key, and darken them using CSS:

```
.basemap-dark { filter: invert(1) hue-rotate(180deg) brightness(0.88) contrast(0.92)
saturate(0.6); }
```

Zoom buttons: our touch display only recognises one finger at a time, so pinch-to-zoom did not work. We enabled Leaflet's built-in zoom buttons and enlarged them to 52 × 52 pixels with CSS so that they are easy to hit with a finger:

```
state.liveMap = L.map("live-map", { zoomControl: true }).setView([state.lat, state.lon], 8);

.leaflet-bar.leaflet-control-zoom a { width: 52px !important; height: 52px !important;
  line-height: 50px !important; font-size: 28px !important; background: rgba(12,17,34,.88) !
important; }
```

Centre button: at the top right of both maps there is a separate button with a crosshair. It is implemented as a Leaflet control and resets the map to the active location and the initial zoom level using `setView` (11 for the forecast map, 8 for the live map). Because the maps are only created when their page is first opened, the code hooks into initForecastMap and initLiveMap and adds the button exactly once:

```
const Center = L.Control.extend({
  options: { position: "topright" },
  onAdd() { /* button with SVG crosshair */ },
});
L.DomEvent.on(a, "click", () => map.setView([state.lat, state.lon], zoom));
```

Change of location: app.js queries /api/location every 10 seconds. If the location has changed, `checkLocation()` moves both maps and their markers with `setView` and `setLatLng` and reloads the outdoor weather and forecast. If the location is changed on the display, standorte.js fires the `wl-location` event, in which case this happens immediately.

8.5 UPDATING THE VALUES

The page queries the values at different intervals, depending on how often they change:

DISPLAYED VALUE	INTERVAL IN THE BROWSER
Indoor values	1 second
Outdoor values	10 seconds

DISPLAYED VALUE	INTERVAL IN THE BROWSER
Pi system values	5 seconds
Clock	1 second
History (while the page is open)	60 seconds
Vorschau (Forecast)	30 minutes
Location (after a change)	10 seconds, immediately on the display

8.6 EXTRAS FOR CONTINUOUS OPERATION

As the display runs around the clock, we added two convenience features:

- **Night mode** Between 22:00 and 6:00 the display is dimmed. A tap wakes it for two minutes. The period can be changed in the configuration, or the feature can be turned off completely.
- **Return to the home page** If nothing has been touched for 90 seconds, the display automatically switches back to the *Innen / Außen* page. This way, the wrong view does not stay on the display permanently after a quick look at another page.

8.7 SETTINGS TAB

Behind the gear icon in the navigation bar is a settings tab. It allows Telegram recipients, the Wi-Fi name and password, locations and the PIN to be changed directly on the display, without SSH and without editing config.py. It was added with the script weatherlab-erweiterung.sh (section 10.8) and extended with weatherlab-standorte.sh (section 8.8).

AREA	CONTENTS
Telegram	approve or reject requests, add, rename, test and remove recipients
PIN	set a new PIN for the tab
WLAN (Wi-Fi)	change the Wi-Fi name (SSID) and Wi-Fi password
Standorte (locations)	search, save, select and remove locations

PIN protection: every request to the tab includes the PIN. settings.py compares it using `hmac.compare_digest` so that the PIN cannot be guessed from the response time. After 5 failed attempts, input is blocked for 120 seconds. When you leave the page, including through the automatic return to the home page, the tab locks itself.

```
def check_pin(given):
    """Returns None if the PIN is correct, otherwise a ready-made error response."""
    ...
    if hmac.compare_digest(str(given or ""), PIN):
        _fails["count"] = 0
        return None
    _fails["count"] += 1
    if _fails["count"] >= MAX_FAILS:          # 5 attempts
        _fails["until"] = now + LOCK_SECONDS  # 120-second lockout
```

One endpoint for everything: the tab only communicates with POST /api/settings. In addition to the PIN, the JSON contains an action, for example `approve`, `add`, `remove`, `test` or `wifi_password`. Without a PIN, only /api/settings/pending is available, so that the orange dot for new requests can appear on the gear icon.

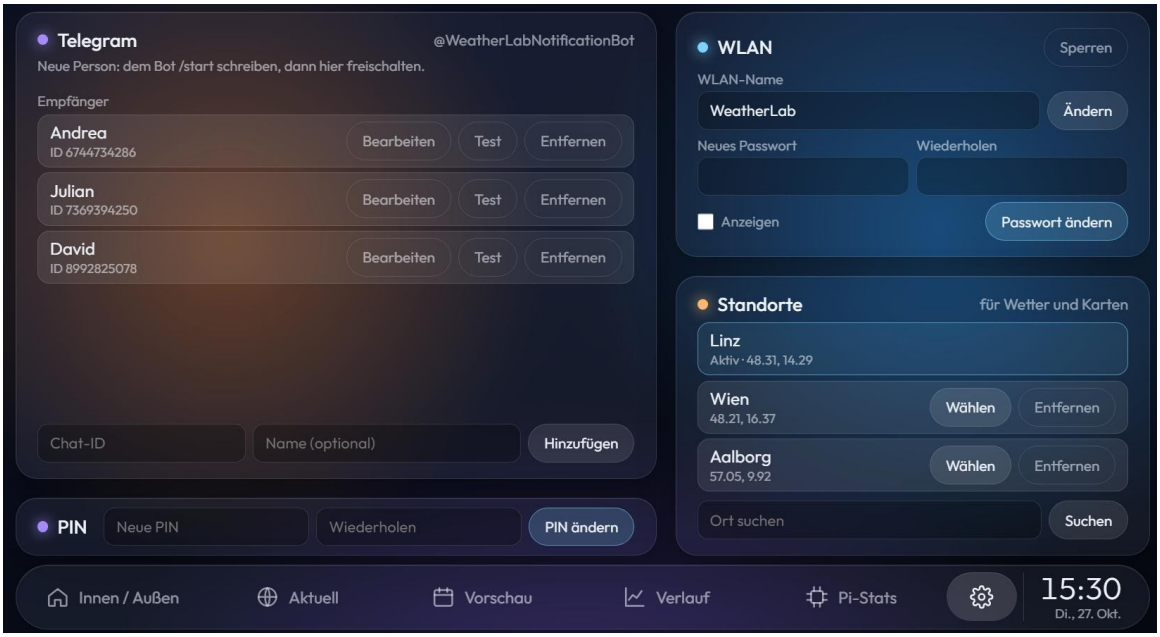
Changing the Wi-Fi: the web application runs without root privileges and is not allowed to write hostapd.conf itself. For this purpose, there is a small helper script, /usr/local/sbin/weatherlab-wlan. A sudo rule in /etc/sudoers.d/weatherlab-wlan allows the service to run exactly this one script:

```
weatherlabadmin ALL=(root) NOPASSWD: /usr/local/sbin/weatherlab-wlan

sudo weatherlab-wlan show                    # shows the Wi-Fi name
echo "NewPassword"   | sudo weatherlab-wlan setpass
echo "NewName"       | sudo weatherlab-wlan setssid
```

The script validates the input again itself (password 8 to 63, name 1 to 32 printable characters), creates a backup hostapd.conf.bak, replaces the `wpa_passphrase=` or `ssid=` line and restarts hostapd after 3 seconds. The delay is needed so that the response still reaches the browser before the Wi-Fi briefly goes down. The new value is passed via standard input rather than as a parameter, so that it does not appear in the process list.

Changing the PIN: the PIN box below Telegram sends the new PIN to /api/settings/pin. locations.py replaces the SETTINGS_PIN line in config.py and also updates the value in the running program, so no restart is necessary.



8.8 LOCATIONS

Originally, the location was fixed in config.py. The script weatherlab-standorte.sh makes it possible to save multiple locations. One of them is active: app.py fetches the outdoor weather and forecast for it, and the maps show it. The data is stored in locations.json:

```
{
  "active": 1,
```

```
  "places": [
    {"name": "Linz", "lat": 48.3069, "lon": 14.2858},
    {"name": "Wien", "lat": 48.2084, "lon": 16.3725}
  ]
}
```

On first start, locations.py creates the file using LAT and LON from config.py. Searching uses the OpenWeather geocoding API, which works with the same API key. The search returns up to five results with state and country, so that places with the same name can be told apart, for example:

```
https://api.openweathermap.org/geo/1.0/direct?q=Wien&limit=5&appid=...
```

Immediate weather update: so that the new forecast is not delayed by up to 30 minutes after a change, weather_loop and forecast_loop no longer wait with `time.sleep` but with `locations.wait()`. This function waits for the same amount of time but returns immediately when the location changes:

```
def wait(seconds):
    """Like time.sleep(seconds), but returns immediately when the location changes."""
    with _cond:
        start = _generation
        _cond.wait_for(lambda: _generation != start, timeout=seconds)
```

Switching without a PIN: on the home page, tapping the location name opens the list of saved locations. This uses the endpoint /api/locations/select, which does not require a PIN. Adding and removing locations is only possible via /api/locations with a PIN. Error messages from the search are filtered so that the API key never reaches the browser.

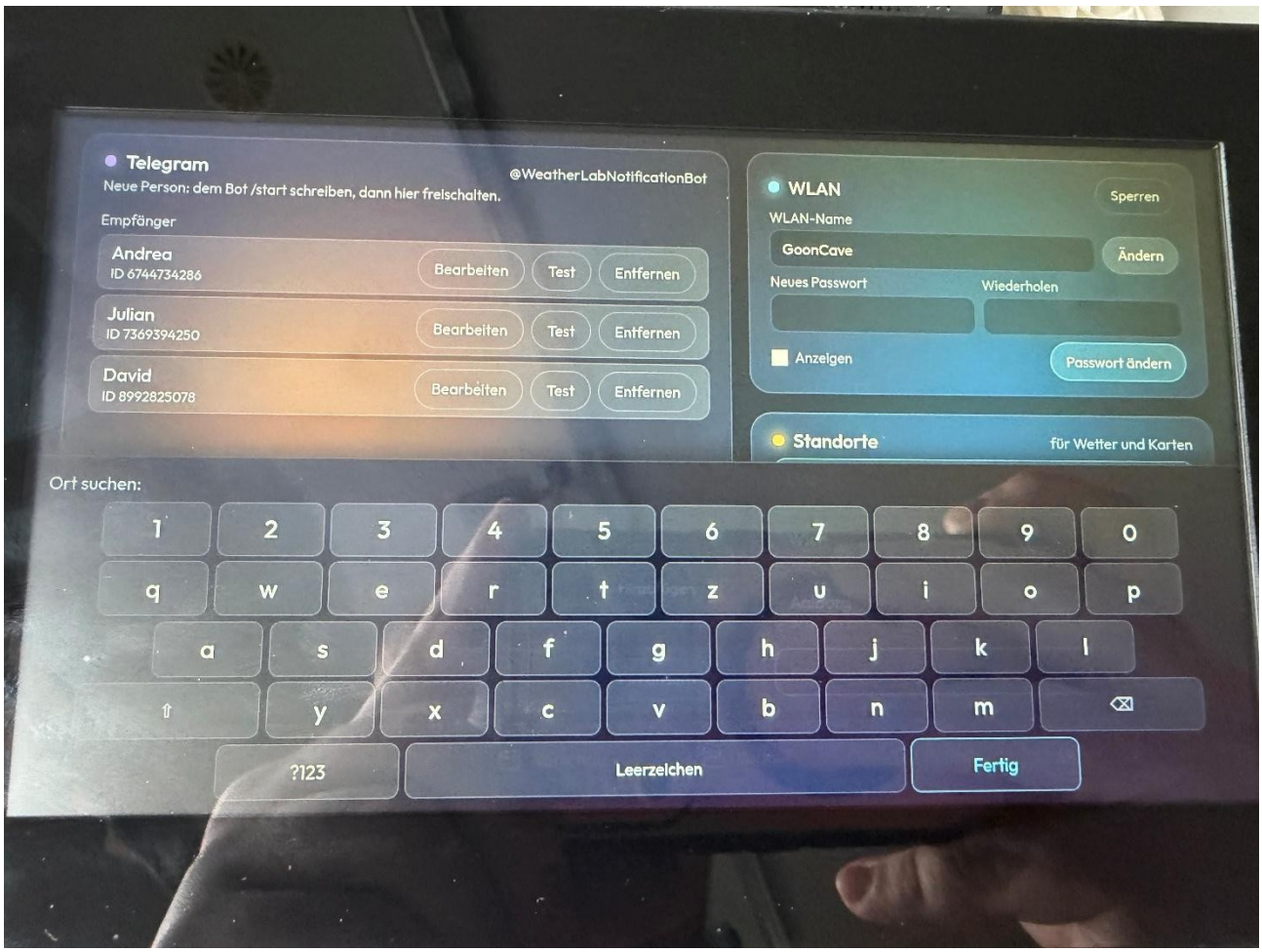
Installation: the extension is installed with `bash ~/pi-dashboard/weatherlab-standorte.sh installieren`. The script creates locations.py and standorte.js, modifies app.py, app.js, settings.js, index.html and style.css at precisely defined points and then checks whether the dashboard responds again. If it cannot find one of these points, it automatically reverts everything (section 11.3). It can be run several times; parts that are already installed are skipped. The PIN change, renaming and the location picker on the home page were later added the same way.

8.9 ON-SCREEN KEYBOARD

To allow names, passwords and places to be entered on the touch display without a physical keyboard, settings.js includes its own on-screen keyboard, which is overlaid at the bottom of the page. It only appears when the page is opened via localhost, i.e. on the display itself. On a phone, the device's normal keyboard is used.

```
if (!["localhost", "127.0.0.1"].includes(location.hostname)) return;
el.setAttribute("inputmode", "none");   // suppress the system keyboard
```

There are three layouts: letters (QWERTZ), special characters and a numeric keypad for chat IDs and the PIN. Above the keys there is a preview of the field, with passwords shown only as dots. When a key is tapped, the keyboard uses `preventDefault` to stop the input field from losing focus. In the location search, Fertig (done) immediately starts the search.


Telegram
@WeatherLabNotificationBot
Neue Person: dem Bot /start schreiben, dann hier freischalten.
Empfänger
Andrea
ID 6744734286
Bearbeiten
Test
Entfernen
Julian
ID 7369394250
Bearbeiten
Test
Entfernen
David
ID 8992825078
Bearbeiten
Test
Entfernen
WLAN
Sperren
WLAN-Name
GoonCave
Ändern
Neues Passwort
Wiederholen
Anzeigen
Passwort ändern
Standorte
für Wetter und Karten
Ort suchen:
1 2 3 4 5 6 7 8 9 0
q w e r t z u i o p
a s d f g h j k l
y x c v b n m
?123
Leerzeichen
Fertig

9. Web Server and Autostart

9.1 SERVICE FOR THE WEB APPLICATION

To make the program start automatically at power-on and restart after a crash, we created a systemd service for it. The file is located at **/etc/systemd/system/pi-dashboard.service**.

```
[Unit]
Description=Pi Dashboard (Flask)
After=network-online.target
Wants=network-online.target

[Service]
User=weatherlabadmin
WorkingDirectory=/home/weatherlabadmin/pi-dashboard
ExecStart=/home/weatherlabadmin/pi-dashboard/venv/bin/python3 /home/weatherlabadmin/pi-
dashboard/app.py
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Here is a line-by-line explanation of each entry:

- **After / Wants=network-online.target** The service only starts once the network is ready.
- **User=weatherlabadmin** The program runs under the normal user account and not as root.
- **WorkingDirectory** Sets the working directory. The history.db database is also stored there.
- **ExecStart** Starts app.py with the Python interpreter from our own environment.
- **Environment=PYTHONUNBUFFERED=1** Ensures that the program's messages appear in the log immediately. Without this line, they remain stuck in the buffer, which initially cost us quite some time when troubleshooting.
- **Restart=always, RestartSec=5** After a crash, the service is restarted automatically after five seconds.

It was enabled and started with:

```
sudo systemctl daemon-reload
sudo systemctl enable --now pi-dashboard.service
sudo systemctl status pi-dashboard.service
```

9.2 APACHE AS A REVERSE PROXY

The web application itself listens on port 5000, so you would always have to enter the address with the port. To make the page available at the usual address without a port, we put Apache in front of it. Apache accepts requests on port 80 and forwards them internally.

First, the required modules are enabled:

```
sudo apt install -y apache2
sudo a2enmod proxy proxy_http
```

The configuration is located at **/etc/apache2/sites-available/pi-dashboard.conf**:

```
<VirtualHost *:80>
    ServerName weatherlab

    ProxyPreserveHost On
    ProxyRequests Off

    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    ErrorLog ${APACHE_LOG_DIR}/pi-dashboard-error.log
    CustomLog ${APACHE_LOG_DIR}/pi-dashboard-access.log combined
</VirtualHost>
```

Here is a line-by-line explanation of each entry:

- **<VirtualHost *:80>** This configuration applies to all requests arriving on port 80.
- **ServerName weatherlab** The hostname under which the server is addressed.
- **ProxyPreserveHost On** Passes the originally requested hostname on to the web application.
- **ProxyRequests Off** Prevents Apache from being misused as an open proxy for other sites.
- **ProxyPass / ProxyPassReverse** Forward all requests to the application on 127.0.0.1:5000 and pass its responses back.
- **ErrorLog / CustomLog** Create separate log files under /var/log/apache2/.

The site is then enabled and the default site disabled:

```
sudo a2ensite pi-dashboard.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

The `configtest` command must report *Syntax OK*. If there is an error, Apache keeps running with the old configuration, so the site remains available.

If the browser later reports the error **503 Service Unavailable**, this means that Apache is running but the web application behind it is not. In this case, check the service log.

9.3 KIOSK MODE ON THE TOUCH DISPLAY

To make the display show the website in full screen automatically after power-on, we added the browser to the autostart. Before this, automatic desktop login must be enabled in `raspi-config` under *System Options*.

The autostart file is located at **~/.config/labwc/autostart**:

```
chromium --kiosk --noerrdialogs --disable-infobars --incognito --disable-session-crashed-bubble
--password-store=basic http://localhost/ &
```

Meaning of the options:

- **--kiosk** Starts the browser in full screen without an address bar or menus.
- **--noerrdialogs** Suppresses error dialogs that would otherwise remain on top of the display.
- **--incognito** No browsing history is stored.
- **--disable-session-crashed-bubble** Suppresses the message that Chromium was not shut down properly, which would otherwise appear after every power cut.
- **--password-store=basic** Chromium then stores passwords without the Raspberry Pi OS keyring. Without this option, a window asking for the keyring password appeared at every startup.
- **http://localhost/** The address displayed. As the browser runs on the Raspberry Pi itself, localhost is sufficient.

Kiosk mode can be exited with `Alt+F4` or via SSH with `pkill chromium`. To restart it via SSH, the display must be specified:

```
DISPLAY=:0 chromium --kiosk http://localhost/ &
```

9.4 OVERVIEW OF THE SERVICES

Once set up completely, the following services run automatically:

SERVICE	TASK
hostapd	broadcasts the Wi-Fi
dnsmasq	assigns IP addresses and resolves names
startup-wifi	sets the static IP and starts the two services above
pi-dashboard	the web application with sensor and weather
apache2	serves the website on port 80
telegram-bot	Telegram bot with alerts and daily report (section 10.7)

Check them all together:

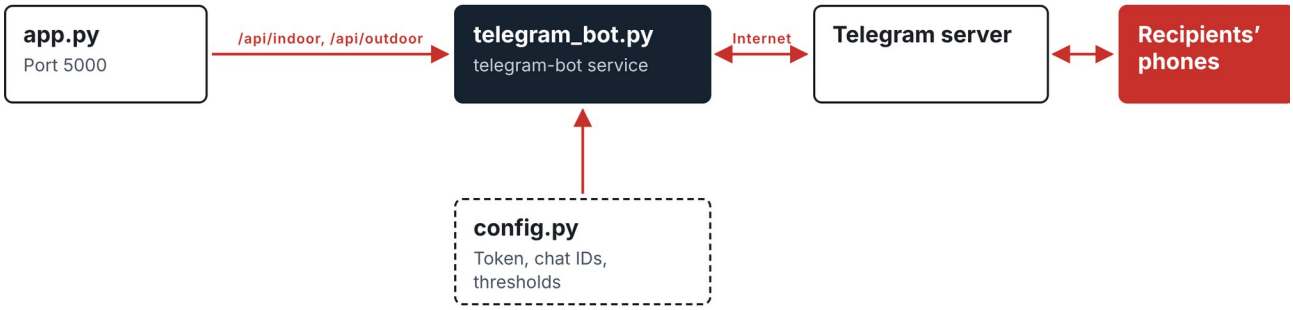
```
systemctl status hostapd dnsmasq startup-wifi pi-dashboard apache2 telegram-bot --no-pager |
grep -E "●|Active"
```

Every service should report *active (running)*. For startup-wifi, *active (exited)* is the correct state, as the script runs once and then exits.

10. Telegram Bot

In the second project phase, we extended the weather station with a Telegram bot. It sends alerts and a daily report to users' phones every morning and responds to commands such as /wetter.

The bot runs as a separate service alongside the web application. It retrieves its values through the same interfaces as the website (`http://127.0.0.1:5000/api/...`), so we did not have to change anything in the working measurement code in app.py.



New files in the project folder and on the system:

FILE	PURPOSE
telegram_bot.py	the bot: commands, alerts, daily report
telegram_state.json	active alerts, date of the last daily report
chat_store.py	shared recipient list for the bot and the settings tab
telegram_chats.json	approved recipients and pending requests
config.py (extended)	token, thresholds, initial value for the chat IDs
telegram-setup.sh	originally set up the bot, now replaced (section 10.8)
/etc/systemd/system/telegram-bot.service	starts the bot at boot

10.1 CREATING THE BOT IN TELEGRAM

A Telegram bot is created via the official bot @BotFather. There, you send `/newbot`, choose a display name and a user name that must end in `bot`. BotFather replies with the token, for example:

```
7412589630:AAHf3kL9xQ2mZpR7vT1wYc8NbE4sJdU6oGi
```

The part before the colon is the bot's ID, the part after it is the secret. Anyone who knows the token can control the bot. It therefore belongs only in config.py and never in screenshots. If it does become public, `/revoke` in BotFather generates a new one. `/setuserpic` and `/setdescription` can also be used to change the bot's profile picture and description.

The token and all bot settings are at the end of **config.py**:

```
TELEGRAM_BOT_TOKEN = "7412589630:AAHf..."
TELEGRAM_CHAT_IDS = [123456789, 987654321]   # initial value only (section 10.4)
TELEGRAM_DAILY_REPORT = "07:00"   # daily report at this time ("" = off)
TELEGRAM_ALERTS = True            # send alerts
TELEGRAM_QUIET_START = "22:00"    # alerts arrive silently during this time
TELEGRAM_QUIET_END = "07:00"

ALERT_TEMP_HIGH = 27        # °C indoors
ALERT_TEMP_LOW = 17         # °C indoors
ALERT_HUMIDITY_HIGH = 65    # % indoors
ALERT_HUMIDITY_LOW = 30     # % indoors
ALERT_PRESSURE_DROP = -3    # hPa in 3 hours
ALERT_FROST_C = 0           # °C outdoors
ALERT_WIND_KMH = 50         # km/h outdoors
ALERT_SENSOR_TIMEOUT = 180  # seconds without new sensor values
ALERT_RAIN = True                       # report rain and snow
ALERT_RAIN_END_MIN = 30                 # dry this long -> all-clear
```

Here is a short explanation of each value:

- **TELEGRAM_BOT_TOKEN:** the key from BotFather. The bot does not start without it.
- **TELEGRAM_CHAT_IDS:** list of chats that receive messages. Since the extension, it is only the initial value: on first start, chat_store.py copies the IDs into telegram_chats.json, after which recipients are managed in the settings tab (section 10.4). Groups have a negative ID.
- **TELEGRAM_DAILY_REPORT:** time of the daily report. An empty value turns it off.
- **TELEGRAM_QUIET_START / END:** during this period, alerts arrive without sound so that nobody is woken up at night.
- **ALERT_...:** the thresholds at which an alert is triggered (section 10.3).
- **ALERT_RAIN / ALERT_RAIN_END_MIN:** turn on the notifications for rain and snow and define how many minutes it must be calm before the all-clear is sent (default 30).

As in app.py, the bot reads every value using `getattr` with a default. If a line is missing in config.py, it still runs. After a change, the bot must be restarted with `sudo systemctl restart telegram-bot`.

10.2 STRUCTURE OF TELEGRAM_BOT.PY

The bot is a single Python program without additional packages. The requests library, which was already installed for OpenWeather, is sufficient for the connection to Telegram. It communicates directly with the Telegram Bot API at `https://api.telegram.org/bot<Token>/<Methode>`.

The bot fetches new messages using **long polling**: it asks with `getUpdates`, and Telegram keeps the request open for up to 20 seconds until a message arrives. This way, commands arrive almost instantly without the bot having to poll every second. The Pi does not need a publicly reachable server for this.

```
def main():
    me = tg("getMe")                        # check token
    tg("setMyCommands", commands=[...])     # command list in the Telegram menu
    state = load_state()
    watch = SensorWatch()
    broadcast("🟢 WeatherLab-Bot ist gestartet. /hilfe zeigt alle Befehle.")
```

```
last_check = 0
while True:
    # 1) Fetch new messages (waits up to 20 s)
    updates = tg_updates(state["offset"])
    for upd in updates:
        state["offset"] = upd["update_id"] + 1
        if "message" in upd:
            handle_message(upd["message"], state)

    # 2) Check alerts and daily report every minute
    if time.time() - last_check >= CHECK_INTERVAL:
        last_check = time.time()
        if CHAT_IDS:
            if ALERTS_ON:
                check_alarms(state, watch)
            check_daily_report(state)
```

Here is a short explanation of the process:

- **offset:** every message has a sequential number. With `offset = last number + 1`, the bot confirms that it has processed it. Otherwise, Telegram would send it again after a restart. The offset is therefore stored in telegram_state.json.
- **check_alarms / check_daily_report:** run at most once per minute between two polls (`TELEGRAM_CHECK_INTERVAL`, default 60 s).
- **Errors:** if Telegram or the internet cannot be reached, the bot waits 10 seconds and tries again. The token is replaced with `***` in the log.

Commands:

COMMAND	REPLY
/wetter	indoor and outdoor values with a ventilation tip
/innen	BME680 readings with trend, daily min/max and pressure trend
/aussen	current weather from OpenWeather
/vorschau	5-day forecast
/verlauf	minimum and maximum of the last 24 hours
/alarme	active alerts and all thresholds
/id	your own chat ID (no longer listed in the menu, but still works)
/hilfe	list of all commands

The replies are formatted with HTML (`parse_mode="HTML"`), for example bold values. All text that comes from the measurement data is first passed through `html.escape`, so that a character such as `<` in a weather description does not break the formatting. Numbers are output with a decimal comma (22,4 °C), as is usual in German.

10.3 ALERTS AND DAILY REPORT

Every minute, the bot fetches `/api/indoor` and `/api/outdoor` and checks all alerts. Each alert has two conditions: one for triggering and one for resetting. The gap between them is called **hysteresis**. Without it, a value hovering right at the threshold (27.0 → 26.9 → 27.0 °C) would trigger a new alert and a new all-clear every minute.

ALERT	SOURCE	TRIGGERED AT	ALL-CLEAR AT
Innen zu warm (too warm indoors)	BME680	≥ 27 °C	< 26 °C
Innen zu kalt (too cold indoors)	BME680	≤ 17 °C	> 18 °C
Luftfeuchte zu hoch (humidity too high)	BME680	≥ 65 %	< 60 %
Luft zu trocken (air too dry)	BME680	≤ 30 %	> 35 %
Schlechte Luft (poor air)	BME680	Schlecht or Sehr schlecht	Gut or Sehr gut
Luftdruck fällt stark (pressure falling sharply)	BME680 + history	≤ −3 hPa in 3 h	> −1.5 hPa in 3 h
Station offline	BME680	no new reading for 180 s	new values again
Frost	OpenWeather	≤ 0 °C	> 1 °C
Starker Wind (strong wind)	OpenWeather	≥ 50 km/h	< 40 km/h
Gewitter (thunderstorm)	OpenWeather	weather icon 11 (thunderstorm)	no thunderstorm for 30 min
Regen (rain)	OpenWeather	weather icon 09 or 10 (rain)	no rain for 30 min
Schnee (snow)	OpenWeather	weather icon 13 (snow)	no snow for 30 min

The values in the table are the defaults from config.py. For air quality, the level “Mittel” (moderate) acts as the hysteresis: the alert remains active until the air is “Gut” (good) again. For the “humidity too high” alert, the bot appends the ventilation recommendation from section 7.3, i.e. whether ventilating helps at the moment.

Rain, snow and thunderstorms: these three alerts have no numerical value with a gap, only the weather icon. As hysteresis, the bot remembers when the event was last reported and only sends the all-clear after 30 minutes without it (ALERT_RAIN_END_MIN). A thunderstorm keeps the rain alert alive, so that “Regen hat aufgehört” (rain has stopped) is not sent in the middle of a thunderstorm.

The alerts are defined in the program as a table (Python dictionary). Each entry has a name, the two conditions and the two message texts:

```
ALARMS = {
    "temp_hoch": {
        "name": "Innen zu warm",
        "an":  lambda i, o: _cmp(_v(i, "temperature"), lambda v: v >= TEMP_HIGH),
        "aus": lambda i, o: _cmp(_v(i, "temperature"), lambda v: v < TEMP_HIGH - 1),
        "text": lambda i, o: f"🔥 <b>Innen zu warm:</b> {num(i['temperature'], ' °C')}",
        "ok":   lambda i, o: f"✅ Innentemperatur wieder normal: {num(i['temperature'], '
°C')}",
    },
    ...
}
```

```
for key, alarm in ALARMS.items():
    if key not in active and alarm["an"](indoor, outdoor):
        active.add(key)
        broadcast(alarm["text"](indoor, outdoor), alarm=True)
    elif key in active and alarm["aus"](indoor, outdoor):
        active.discard(key)
        broadcast(alarm["ok"](indoor, outdoor))
```

`i` stands for the indoor values, `o` for the outdoor values. If a reading is missing, `_cmp` returns `None` and the state of the alert remains unchanged. A brief OpenWeather outage therefore does not trigger a false all-clear. The currently active alerts are stored in telegram_state.json, so the bot does not resend all alerts after a restart.

Station offline: the indoor values carry a time stamp (`timestamp`). The bot remembers when it last changed. If it stays the same for more than 180 seconds or the dashboard cannot be reached, a warning is sent together with the command for troubleshooting.

Quiet hours: between 22:00 and 07:00, the bot sends alerts with `disable_notification`. They arrive, but without sound. All-clear messages and the daily report always arrive normally with sound.

Daily report: at 07:00 the bot sends the indoor values, outdoor values, ventilation tip and forecast. If the Pi was switched off at that time, it catches up with the report within the next hour. The date of the last report is stored in telegram_state.json, so that it is only sent once a day.

10.4 APPROVING RECIPIENTS

Not everyone who finds the bot in Telegram should receive readings and alerts. Initially, the permitted chat IDs were stored in config.py, and the bot had to be restarted for every new person. Since the extension (section 10.8), the bot and the settings tab manage the recipients together in the file telegram_chats.json:

```
{"chats":   [{"id": 123456789, "name": "David"}],
 "pending": [{"id": 987654321, "name": "Hr. Mayr", "ts": 1760350000}]}
```

chats contains the approved recipients, pending the requests. Both programs access the file via chat_store.py. To prevent the bot and the website from overwriting each other's changes, chat_store.py locks the file while writing using `fcntl.flock`. On the very first start, it takes over the IDs from TELEGRAM_CHAT_IDS.

If an unknown chat sends /start, it is added to the list of requests and receives a reply with its chat ID:

```
if not allowed:
    if command in ("start", "id"):
        chat_store.add_pending(chat_id, chat_name(chat))
        send(chat_id, f"👋 Hallo! Deine Anfrage ist angekommen.\n"
                      f"Bitte jetzt am Display der Wetterstation unter "
                      f"<b>Einstellungen → Telegram</b> auf <b>Freischalten</b> tippen. ...")
    else:
        send(chat_id, "⛔ Dieser Chat ist noch nicht freigeschaltet. Schreib /start.")
    return
```

An orange dot then appears on the gear icon. In the settings tab, the request is accepted with Freischalten (approve). The bot reads the list again for every message, so a restart is no longer necessary. The person immediately receives the message “Du bist jetzt für die WeatherLab-Wetterstation freigeschaltet!” (you are now approved for the WeatherLab weather station).

In the tab, recipients can also be added manually by chat ID, renamed with Bearbeiten (edit), checked with Test and deleted with Entfernen (remove). When approving, the bot takes the name from Telegram, i.e. first and last name or the group name. A changed name is saved via /api/settings/rename_chat and is no longer overwritten by the bot afterwards.

10.5 THE TELEGRAM-BOT SERVICE

To make the bot start at boot and restart after a crash, it runs as a separate systemd service. Path: **/etc/systemd/system/telegram-bot.service**

```
[Unit]
Description=WeatherLab Telegram-Bot
After=network-online.target pi-dashboard.service
Wants=network-online.target

[Service]
User=weatherlabadmin
WorkingDirectory=/home/weatherlabadmin/pi-dashboard
ExecStart=/home/weatherlabadmin/pi-dashboard/venv/bin/python3 /home/weatherlabadmin/pi-
dashboard/telegram_bot.py
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Here is a short explanation of the entries:

- **After=... pi-dashboard.service:** the bot starts after the web application, because it gets its values from it.
- **User=weatherlabadmin:** the bot runs without root privileges.
- **ExecStart:** starts the bot with the Python interpreter from the virtual environment in which requests is installed.
- **PYTHONUNBUFFERED=1:** output appears in the log immediately. Without this line, Python holds it back and `journalctl` shows nothing for a long time.
- **Restart=always / RestartSec=10:** after a crash, the bot restarts after 10 seconds.

The service is enabled and checked as follows:

```
sudo systemctl daemon-reload
sudo systemctl enable --now telegram-bot
sudo journalctl -u telegram-bot -f          # follow the log live
```

At startup, the bot writes for example `Bot @weatherlab_linz_bot gestartet, freigeschaltete Chats: [123456789]` to the log and sends “WeatherLab-Bot ist gestartet” (WeatherLab bot started) to all recipients. This also shows when the Pi has been restarted.

10.6 SETUP WITH TELEGRAM-SETUP.SH

The bot is set up with the script **telegram-setup.sh**. It contains telegram_bot.py and creates the file itself, so that only one file has to be copied to the Pi. It can be transferred with FileZilla, for example (SFTP protocol, port 22, user weatherlabadmin), and placed in the project folder. The script is run as a normal user, not with sudo:

```
bash ~/pi-dashboard/telegram-setup.sh
```

The script performs these steps:

1. **Create telegram_bot.py** and check it for errors with `py_compile`.
2. **Stop the running bot**, so that it does not intercept the /start message in the next step.
3. **Ask for the token** and verify it with Telegram using `getMe`. If a token is already configured, it can be kept by pressing Enter.
4. **Wait for /start:** for up to 2 minutes, the script polls Telegram for new messages and records the chat IDs. Older messages are skipped beforehand.
5. **Extend config.py:** a backup `config.py.bak-telegram` is created first. If the Telegram section is missing, it is appended with the default values. New chat IDs are added to the list, existing ones are kept. If config.py is faulty afterwards, the script restores the backup.
6. **Set up the service:** write telegram-bot.service (section 10.5), enable and start it. Finally, the script shows the last lines of the log.

The script also removes Windows line endings itself. These easily occur when copying from a Windows PC and had caused the “Exec format error” with startupWIFI.sh. It can be run as often as required, for example to approve further chats or to enter a new token.

In the meantime, telegram-setup.sh has been replaced by weatherlab-erweiterung.sh (section 10.8). The new script contains the new bot and the settings tab, so telegram-setup.sh is no longer needed.

10.7 TESTING WITHOUT REAL TELEGRAM

Before installing the extension on the Pi, we tested it on a computer without a sensor and without Telegram. For this, two addresses in config.py can be changed, which otherwise keep their default values:

```
TELEGRAM_API_URL = "http://127.0.0.1:8765"   # instead of https://api.telegram.org
DASHBOARD_URL = "http://127.0.0.1:8765"      # instead of http://127.0.0.1:5000
```

A small Python test server ran at this address. It returned fixed readings such as `/api/indoor` and also emulated Telegram: it answered `getMe` and `getUpdates` and recorded every `sendMessage` message. This made it possible to check what the bot would send.

TEST	RESULT
all commands from an approved chat	correct replies, numbers with decimal comma
/start from an unknown chat	reply with the chat ID, no readings
thresholds exceeded	one alert each, no duplicate on the second run

TEST	RESULT
value just below the threshold (hysteresis)	no all-clear
values back to normal	all-clear for every alert
sensor time stamp unchanged for 16 min	warning “Wetterstation meldet sich nicht” (station not responding)
dashboard not reachable	message “Wetterstation ist gerade nicht erreichbar” (station currently unreachable)
daily report triggered twice	only one report
setup script with test server	token verified, chat ID added, old message skipped

10.8 THE EXTENSION SCRIPT

In the third project phase, we extended the bot and the website with a single script. It is placed in the project folder via FileZilla or by pasting it into PuTTY and is run as a normal user, not with sudo:

```
bash ~/pi-dashboard/weatherlab-erweiterung.sh installieren
```

The script performs these steps:

1. Ask for the PIN for the settings tab (4 to 8 digits) and keep the existing bot token or enter a new one. The token is verified with Telegram using getMe.
2. Create a backup: project and system files are saved to backups/vor-erweiterung-DATUM.tar.gz (section 11.3).
3. Create the new files: chat_store.py, settings.py, the new telegram_bot.py and static/settings.js. All Python files are checked with py_compile.
4. Add the gear icon and the settings page to index.html, style.css and app.py, and extend config.py with the PIN and the Telegram section.
5. Install the Wi-Fi helper script and the sudo rule. The rule is checked with visudo beforehand.
6. Restart the dashboard, check the response and start the telegram-bot service. If anything goes wrong, the script offers to revert everything.

New in the bot are the rain and snow alerts (section 10.3) and approval via the settings tab (section 10.4). Like telegram-setup.sh, this script also removes Windows line endings itself.

11. Operation and Troubleshooting

11.1 APPLYING CHANGES

Depending on which file was changed, different steps are required:

CHANGED FILE	REQUIRED AFTERWARDS
config.py or app.py	sudo systemctl restart pi-dashboard
index.html, style.css, app.js	Ctrl+F5 in the browser, sudo reboot for the display
settings.py, locations.py, chat_store.py	sudo systemctl restart pi-dashboard
telegram_bot.py	sudo systemctl restart telegram-bot
settings.js, standorte.js	Ctrl+F5 in the browser, sudo reboot for the display
pi-dashboard.service	sudo systemctl daemon-reload, then restart
hostapd.conf	sudo systemctl restart hostapd
dnsmasq.conf	sudo systemctl restart dnsmasq, reconnect the device
Apache configuration	configtest, then sudo systemctl reload apache2

11.2 USEFUL COMMANDS

```
journalctl -u pi-dashboard -f                  # follow the log live
journalctl -xeu pi-dashboard --no-pager | tail -30
journalctl -u telegram-bot -f                  # bot log
curl -s http://localhost/api/indoor            # check indoor values
curl -s http://localhost/api/outdoor           # check outdoor values
i2cdetect -y 1                                 # is the sensor visible?
ip addr show wlan0                             # does wlan0 have the static IP?
nmcli device status                            # is wlan0 unmanaged?
curl -s http://localhost/api/locations          # saved locations
sudo /usr/local/sbin/weatherlab-wlan show       # current Wi-Fi name
cat ~/pi-dashboard/telegram_chats.json          # Telegram recipients
```

11.3 BACKUP

The most important part is the project folder. It can be copied from the Windows PC as follows:

```
scp -r weatherlabadmin@192.168.10.1:~/pi-dashboard C:\Backup\
```

The system files from chapter 3 can be backed up as well:

```
sudo tar czf ~/konfig-backup.tar.gz /etc/hostapd/hostapd.conf /etc/dnsmasq.conf
/etc/dnsmasq.d/ /etc/NetworkManager/conf.d/unmanaged.conf /scripts/startupWIFI.sh
/etc/systemd/system/startup-wifi.service /etc/systemd/system/pi-dashboard.service
/etc/apache2/sites-available/pi-dashboard.conf /etc/iptables/rules.v4
```

Automatic backups by the scripts: weatherlab-erweiterung.sh and weatherlab-standorte.sh back up all affected files to the backups folder in the project folder before every change. If

something goes wrong during installation, they restore the backup automatically. To do it manually:

```
bash ~/pi-dashboard/weatherlab-standorte.sh zurueck     # locations, PIN change, renaming
bash ~/pi-dashboard/weatherlab-erweiterung.sh zurueck   # settings tab and new bot
```

weatherlab-standorte.sh uses the latest backup, while weatherlab-erweiterung.sh lets you choose one and asks whether the Wi-Fi should be reset as well. Because every update creates a new backup, we moved older backups and old .bak files to a separate folder, ~/weatherlab-archiv. We keep the oldest vor-standorte backup and the vor-erweiterung backup, which allow you to return to the state before each extension.

11.4 PROBLEMS AND SOLUTIONS

Several problems occurred during setup. We record them here together with their solutions, as experience shows that they tend to recur when the setup is rebuilt.

PROBLEM	CAUSE	SOLUTION
Unit hostapd.service is masked	hostapd is masked after installation	sudo systemctl unmask hostapd
hostapd does not start, unknown configuration item	incomplete lines ieee80211n and ieee80211d	lines removed, checked with sudo hostapd -dd
NetworkManager no longer starts	missing [keyfile] section and wrong separator in unmanaged.conf	file corrected, then systemctl reset-failed
device does not get an IP address	wlan0 had no static address	run startupWIFI.sh or set it up as a service
connecting takes a very long time	unreachable DNS server entered in dnsmasq.conf	replaced with 1.1.1.1 and 8.8.8.8
Wi-Fi available, but no internet	ip_forward was reset after every reboot	value made permanent in /etc/sysctl.d/99-wetterstation.conf
Exec format error in the script	Windows line breaks, missing first line	dos2unix and added #!/bin/bash
no sensor values, only dashes	the constant ENABLE_GAS_MEASUREMENT does not exist in this version	both spellings handled
outdoor values stay empty, Invalid API key	new key not yet activated	wait up to two hours
503 Service Unavailable	Apache is running, the web application is not	check the log, fix the syntax error in app.py
lgpio cannot be installed	package is built from source, swig is missing	system package python3-lgpio used
map on the touch display cannot be zoomed	the display only recognises one finger, pinch-to-zoom does not work	Leaflet zoom buttons enabled and enlarged
keyring prompt at every	Chromium wants to unlock the	--password-store=basic

PROBLEM	CAUSE	SOLUTION
startup	password store	added to the autostart line
new feature not visible on the PC	browser shows app.js from its cache	Ctrl+F5, sudo reboot for the display
command in PuTTY reports event not found	the exclamation mark is interpreted by the shell as a history command	run set +H before pasting

11.5 NOTES ON CONTINUOUS OPERATION

- After it is first switched on, the gas sensor needs 24 to 48 hours before the air quality values are stable. The thresholds should then be adjusted to the room.
- After a fresh installation, the history chart is initially empty. The first lines appear after about ten minutes, the complete 24-hour view after one day.
- During the first three hours, the pressure trend shows *Tendenz wird ermittelt* (trend being determined), because the comparison value first has to be collected.
- After a restart, it takes one to two minutes before the air quality, and with it the status LEDs, show a value again.
- Every update makes the extension scripts create a new backup in backups. Older backups should be moved to the archive from time to time (section 11.3).

11.6 CONCLUSION

The project combines several fields: networking for the access point, electronics for the sensor and the LEDs, programming for the web application, and server administration for Apache and the services.

What we found particularly instructive was that most errors were not in the actual program code but in small configuration details: a missing colon, a truncated line, an invisible character at the end of a line, or a setting that was lost after a reboot. Systematic checking using the logs and the test commands proved to be the fastest way to a solution.

The finished system runs on its own: after power-on, the access point starts, measurement begins, the website is served and the display shows the values, without anyone having to enter anything.

With the Telegram bot, the station can also be used remotely: it gets in touch on its own when it becomes too warm or too humid indoors, or when frost, storms or a thunderstorm are on the way outside.

12. Appendix: app.py Source Code

This appendix contains the complete program code of the file **app.py**. For better readability, it is divided into individual sections, each preceded by an explanation. The comments and messages in the source code are in German, as in the original program.

12.1 HEADER AND SETTINGS

First, the required libraries are imported. The import of config.py is wrapped in a check: if the file is missing, the program exits with a clear message instead of crashing with an obscure error. The import of gpiozero is also safeguarded so that the weather station keeps running without LEDs.

The settings from config.py are read using `getattr` with a default value. This means the program also works with an older configuration file that does not yet contain the newer entries.

```
"""
Pi Dashboard – Innen (BME680) & Außen (OpenWeather) im Webinterface.

Start:
    python3 app.py
Dann im Browser (bzw. Kiosk-Modus auf dem Touch Display):
    http://localhost:5000
"""

import os
import sqlite3
import subprocess
import threading
import time

import psutil
import requests
from flask import Flask, jsonify, render_template, request

try:
    import config
except ImportError:
    raise SystemExit(
        "config.py fehlt! Bitte config.py.example nach config.py kopieren "
        "und die Werte (API-Key, Standort) eintragen."
    )

import bme680

try:
    from gpiozero import LED
except Exception:  # Bibliothek fehlt oder kein GPIO vorhanden
    LED = None

app = Flask(__name__)
from settings import settings_bp  # Einstellungen-Tab
app.register_blueprint(settings_bp)
import locations  # Standorte
app.register_blueprint(locations.bp)

# Neue Einstellungen mit Standardwerten, damit eine ältere config.py weiter funktioniert
HISTORY_INTERVAL = getattr(config, "HISTORY_INTERVAL", 60)
HISTORY_DAYS = getattr(config, "HISTORY_DAYS", 7)
```

```
NIGHT_START = getattr(config, "NIGHT_START", 22)
NIGHT_END = getattr(config, "NIGHT_END", 6)
IDLE_RETURN_SECONDS = getattr(config, "IDLE_RETURN_SECONDS", 90)
LED_ENABLED = getattr(config, "LED_ENABLED", True)
LED_PIN_GRUEN = getattr(config, "LED_PIN_GRUEN", 17)
LED_PIN_GELB = getattr(config, "LED_PIN_GELB", 27)
LED_PIN_ROT = getattr(config, "LED_PIN_ROT", 22)

DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "history.db")
```

12.2 SHARED DATA STORES

Three dictionaries are created here to hold the most recent readings. The background loops write to them and the interfaces read from them.

Each store has a so-called lock. As several threads run at the same time, it prevents data being written and read simultaneously. Without this protection, the website could pick up a partially updated data set.

```
sensor_lock = threading.Lock()
sensor_data = {
    "temperature": None,
    "humidity": None,
    "altitude": None,
    "pressure": None,
    "air_quality": None,
    "timestamp": None,
}

weather_lock = threading.Lock()
weather_data = {
    "temperature": None,
    "feels_like": None,
    "humidity": None,
    "pressure": None,
    "wind_speed": None,
    "description": None,
    "icon": None,
    "city": None,
    "sunrise": None,
    "sunset": None,
    "timestamp": None,
}

forecast_lock = threading.Lock()
forecast_data = {"days": [], "timestamp": None}
```

12.3 DATABASE

`init_db` creates the table if it does not exist yet. It also checks whether the column for the air pressure exists. We added this check when we added the air pressure later, so that existing databases do not have to be deleted.

`history_loop` runs permanently in the background and writes one row every minute. Entries older than eight days are then deleted. If neither indoor nor outdoor values are available, the run is skipped.

```
def db_connect():
    return sqlite3.connect(DB_PATH, timeout=5)
```

```
def init_db():
    with db_connect() as conn:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS readings ("
            "ts INTEGER PRIMARY KEY, in_temp REAL, in_hum REAL, out_temp REAL, out_hum REAL)"
        )
        spalten = [z[1] for z in conn.execute("PRAGMA table_info(readings)")]
        if "pressure" not in spalten:
            conn.execute("ALTER TABLE readings ADD COLUMN pressure REAL")
            print("Datenbank um Spalte pressure erweitert")


def history_loop():
    """Speichert regelmäßig einen Messpunkt und löscht alte Einträge."""
    while True:
        time.sleep(HISTORY_INTERVAL)
        try:
            with sensor_lock:
                in_temp = sensor_data["temperature"]
                in_hum = sensor_data["humidity"]
                druck = sensor_data["pressure"]
            with weather_lock:
                out_temp = weather_data["temperature"]
                out_hum = weather_data["humidity"]
            if in_temp is None and out_temp is None:
                continue
            now = int(time.time())
            with db_connect() as conn:
                conn.execute(
                    "INSERT OR REPLACE INTO readings (ts, in_temp, in_hum, out_temp, out_hum,
pressure) "
                    "VALUES (?, ?, ?, ?, ?, ?)",
                    (now, in_temp, in_hum, out_temp, out_hum, druck),
                )
                conn.execute(
                    "DELETE FROM readings WHERE ts < ?", (now - (HISTORY_DAYS + 1) * 86400,)
                )
        except Exception as exc:
            print("Verlauf-Fehler:", exc)
```

12.4 EVALUATING THE HISTORY

`pressure_trend` compares the current air pressure with the average from about three hours ago. Rather than a single old reading, the average over half an hour is used, so that individual outliers do not distort the result.

`indoor_extras` calculates the daily values and the temperature trend. The daily minimum and maximum are calculated from midnight; the trend is derived from the comparison with the value from 30 minutes ago.

```
PRESSURE_TREND_TEXT = {
    "stark_steigend": "stark steigend",
    "steigend": "steigend",
    "gleich": "gleichbleibend",
    "fallend": "fallend",
    "stark_fallend": "stark fallend",
}


def pressure_trend(current_pressure):
    """Vergleicht den aktuellen Luftdruck mit dem Wert von vor 3 Stunden."""
    if current_pressure is None:
        return None, None, None
    try:
        now = time.time()
```

```
        with db_connect() as conn:
            (frueher,) = conn.execute(
                "SELECT AVG(pressure) FROM readings WHERE ts BETWEEN ? AND ? AND pressure IS NOT
NULL",
                (now - 3.5 * 3600, now - 2.5 * 3600),
            ).fetchone()
        if frueher is None:
            return None, None, None
        diff = round(current_pressure - frueher, 1)
        if diff >= 3:
            stufe = "stark_steigend"
        elif diff >= 1:
            stufe = "steigend"
        elif diff <= -3:
            stufe = "stark_fallend"
        elif diff <= -1:
            stufe = "fallend"
        else:
            stufe = "gleich"
        return stufe, PRESSURE_TREND_TEXT[stufe], diff
    except Exception as exc:
        print("Drucktendenz-Fehler:", exc)
        return None, None, None


def indoor_extras(current_temp):
    """Tages-Min/Max und Trend der Innentemperatur aus dem Verlauf."""
    extras = {"today_min": None, "today_max": None, "trend": None}
    try:
        now = time.time()
        lt = time.localtime(now)
        midnight = time.mktime((lt.tm_year, lt.tm_mon, lt.tm_mday, 0, 0, 0, 0, 0, -1))
        with db_connect() as conn:
            mn, mx = conn.execute(
                "SELECT MIN(in_temp), MAX(in_temp) FROM readings WHERE ts >= ? AND in_temp IS
NOT NULL",
                (midnight,),
            ).fetchone()
            (past,) = conn.execute(
                "SELECT AVG(in_temp) FROM readings WHERE ts BETWEEN ? AND ? AND in_temp IS NOT
NULL",
                (now - 35 * 60, now - 25 * 60),
            ).fetchone()
        values = [v for v in (mn, mx, current_temp) if v is not None]
        if values:
            extras["today_min"] = round(min(values), 1)
            extras["today_max"] = round(max(values), 1)
        if past is not None and current_temp is not None:
            diff = current_temp - past
            extras["trend"] = "up" if diff > 0.3 else "down" if diff < -0.3 else "flat"
    except Exception as exc:
        print("Extras-Fehler:", exc)
    return extras
```

12.5 STATUS LEDS

`init_leds` registers the three LEDs at startup. If this fails, for example because the library is not available, the list is cleared and the program continues without LEDs.

`update_leds` assigns a colour to the air quality and switches off the other LEDs, so that exactly one is always lit.

```
leds = {}


def init_leds():
```

```
    """Legt die drei LEDs an. Ohne gpiozero oder GPIO laeuft alles ohne LEDs weiter."""
    if not LED_ENABLED or LED is None:
        return
    try:
        leds["gruen"] = LED(LED_PIN_GRUEN)
        leds["gelb"] = LED(LED_PIN_GELB)
        leds["rot"] = LED(LED_PIN_ROT)
        print("Status-LEDs bereit: GPIO", LED_PIN_GRUEN, LED_PIN_GELB, LED_PIN_ROT)
    except Exception as exc:
        leds.clear()
        print("LED-Fehler, laeuft ohne LEDs weiter:", exc)


def update_leds(quality):
    """Sehr gut/Gut -> gruen, Mittel -> gelb, Schlecht/Sehr schlecht -> rot."""
    if not leds:
        return
    farbe = {
        "Sehr gut": "gruen",
        "Gut": "gruen",
        "Mittel": "gelb",
        "Schlecht": "rot",
        "Sehr schlecht": "rot",
    }.get(quality)
    try:
        for name, led in leds.items():
            led.on() if name == farbe else led.off()
    except Exception as exc:
        print("LED-Fehler:", exc)
```

12.6 SENSOR

`init_sensor` tries both possible I2C addresses. Our board responds at 0x77, others at 0x76. If both fail, a clear error is raised.

`sensor_loop` keeps trying to find the sensor at startup until it succeeds. After that, it measures at the configured interval. The air quality value is only accepted when the sensor reports `heat_stable`, i.e. the heater has reached its temperature.

```
def compute_altitude(pressure_hpa):
    """Höhe (m) aus Luftdruck über die internationale barometrische Höhenformel."""
    return round(
        44330 * (1 - (pressure_hpa / config.SEA_LEVEL_PRESSURE_HPA) ** (1 / 5.255)), 1
    )


def gas_quality_label(gas_resistance):
    """Ordnet den rohen BME680-Gaswiderstand (Ohm) einer Textkategorie zu."""
    if gas_resistance is None:
        return None
    t = config.GAS_QUALITY_THRESHOLDS
    if gas_resistance >= t["sehr_gut"]:
        return "Sehr gut"
    if gas_resistance >= t["gut"]:
        return "Gut"
    if gas_resistance >= t["mittel"]:
        return "Mittel"
    if gas_resistance >= t["schlecht"]:
        return "Schlecht"
    return "Sehr schlecht"


# Je nach Version der Bibliothek heisst die Konstante anders
ENABLE_GAS = getattr(bme680, "ENABLE_GAS_MEASUREMENT", getattr(bme680, "ENABLE_GAS_MEAS", 1))
```

```
def init_sensor():
    """BME680 initialisieren. Probiert beide I2C-Adressen (0x76 und 0x77)."""
    letzter_fehler = None
    sensor = None
    for adresse in (bme680.I2C_ADDR_PRIMARY, bme680.I2C_ADDR_SECONDARY):
        try:
            sensor = bme680.BME680(adresse)
            break
        except Exception as exc:
            letzter_fehler = exc
    if sensor is None:
        raise RuntimeError(f"BME680 auf 0x76 und 0x77 nicht erreichbar: {letzter_fehler}")

    sensor.set_humidity_oversample(bme680.OS_2X)
    sensor.set_pressure_oversample(bme680.OS_4X)
    sensor.set_temperature_oversample(bme680.OS_8X)
    sensor.set_filter(bme680.FILTER_SIZE_3)
    sensor.set_gas_status(ENABLE_GAS)
    sensor.set_gas_heater_temperature(320)
    sensor.set_gas_heater_duration(150)
    sensor.select_gas_heater_profile(0)
    return sensor


def sensor_loop():
    while True:
        try:
            sensor = init_sensor()
            break
        except Exception as exc:
            print("BME680 nicht gefunden, neuer Versuch in 10 s:", exc)
            time.sleep(10)
    while True:
        try:
            if sensor.get_sensor_data():
                with sensor_lock:
                    sensor_data["temperature"] = round(sensor.data.temperature, 1)
                    sensor_data["humidity"] = round(sensor.data.humidity, 1)
                    sensor_data["altitude"] = compute_altitude(sensor.data.pressure)
                    sensor_data["pressure"] = round(sensor.data.pressure, 1)
                    if sensor.data.heat_stable:
                        sensor_data["air_quality"] =
gas_quality_label(sensor.data.gas_resistance)
                        update_leds(sensor_data["air_quality"])
                    sensor_data["timestamp"] = time.strftime("%H:%M:%S")
        except Exception as exc:
            print("BME680-Fehler:", exc)
        time.sleep(config.SENSOR_INTERVAL)
```

12.7 QUERYING OPENWEATHER

Both loops build the URL with the coordinates and the key and evaluate the response. The parameter `units=metric` gives degrees Celsius, `lang=de` German weather descriptions.

The coordinates come from `locations.current()`, i.e. from the selected location (section 8.8). Instead of time.sleep, the loops wait with `locations.wait()`, so that the new weather is fetched immediately after a change of location.

The wind speed is converted from metres per second to kilometres per hour with a factor of 3.6. OpenWeather supplies the sunrise and sunset times as time stamps, which we convert into a readable time.

For the forecast, OpenWeather supplies values in three-hour steps. We group them by date, take the minimum and maximum for each day and use the midday entry for the icon and description, as it best describes the day's weather.

```
def weather_loop():
    while True:
        try:
            url = (
                "https://api.openweathermap.org/data/2.5/weather"
                f"?lat={locations.current()['lat']}&lon={locations.current()['lon']}"
                f"&appid={config.OPENWEATHER_API_KEY}&units=metric&lang=de"
            )
            resp = requests.get(url, timeout=10)
            data = resp.json()

            if resp.status_code != 200:
                print("OpenWeather-Fehler:", data.get("message"))
            else:
                with weather_lock:
                    weather_data["temperature"] = round(data["main"]["temp"], 1)
                    weather_data["feels_like"] = round(data["main"]["feels_like"], 1)
                    weather_data["humidity"] = data["main"]["humidity"]
                    weather_data["pressure"] = data["main"].get("pressure")
                    weather_data["wind_speed"] = round(data["wind"]["speed"] * 3.6, 1)
                    weather_data["description"] = data["weather"][0]["description"]
                    weather_data["icon"] = data["weather"][0]["icon"]
                    weather_data["city"] = data["name"]
                    weather_data["sunrise"] = time.strftime("%H:%M", time.localtime(data["sys"]
["sunrise"]))
                    weather_data["sunset"] = time.strftime("%H:%M", time.localtime(data["sys"]
["sunset"]))
                    weather_data["timestamp"] = time.strftime("%H:%M:%S")
        except Exception as exc:
            print("Wetter-Fehler:", exc)
        locations.wait(config.WEATHER_INTERVAL)


def forecast_loop():
    """Holt die 5-Tage-Vorhersage (3h-Werte) und fasst sie zu Tageswerten zusammen."""
    while True:
        try:
            url = (
                "https://api.openweathermap.org/data/2.5/forecast"
                f"?lat={locations.current()['lat']}&lon={locations.current()['lon']}"
                f"&appid={config.OPENWEATHER_API_KEY}&units=metric&lang=de"
            )
            resp = requests.get(url, timeout=10)
            data = resp.json()

            if resp.status_code != 200:
                print("Vorhersage-Fehler:", data.get("message"))
            else:
                days = {}
                for entry in data["list"]:
                    date, hour = entry["dt_txt"].split(" ")
                    day = days.setdefault(date, {"temps": [], "icon": None, "desc": None})
                    day["temps"].append(entry["main"]["temp"])
                    if hour == "12:00:00" or day["icon"] is None:
                        day["icon"] = entry["weather"][0]["icon"]
                        day["desc"] = entry["weather"][0]["description"]

                day_list = []
                for date, day in list(days.items())[:5]:
                    day_list.append(
                        {
                            "date": date,
                            "min": round(min(day["temps"]), 1),
                            "max": round(max(day["temps"]), 1),
                            "icon": day["icon"],
```

```
                            "description": day["desc"],
                        }
                    )
                with forecast_lock:
                    forecast_data["days"] = day_list
                    forecast_data["timestamp"] = time.strftime("%H:%M:%S")
        except Exception as exc:
            print("Vorhersage-Fehler:", exc)
        locations.wait(config.FORECAST_INTERVAL)
```

12.8 RASPBERRY PI SYSTEM VALUES

The CPU temperature is read using the `vcgencmd` program, which is part of Raspberry Pi OS. If that does not work, the system file under /sys is read instead. The other values are provided by the psutil library.

```
def get_pi_stats():
    """Liest CPU-Temperatur, Auslastung, RAM, Speicherplatz und Laufzeit aus."""
    temp = None
    try:
        out = subprocess.check_output(["vcgencmd", "measure_temp"]).decode()
        temp = float(out.strip().replace("temp=", "").replace("'C", ""))
    except Exception:
        try:
            with open("/sys/class/thermal/thermal_zone0/temp") as f:
                temp = round(int(f.read().strip()) / 1000, 1)
        except Exception:
            temp = None

    cpu_percent = psutil.cpu_percent(interval=0.3)
    mem = psutil.virtual_memory()
    disk = psutil.disk_usage("/")
    uptime_hours = round((time.time() - psutil.boot_time()) / 3600, 1)

    return {
        "cpu_temp": temp,
        "cpu_percent": cpu_percent,
        "ram_percent": mem.percent,
        "ram_used_mb": round(mem.used / 1024 / 1024),
        "ram_total_mb": round(mem.total / 1024 / 1024),
        "disk_percent": disk.percent,
        "disk_used_gb": round(disk.used / 1024**3, 1),
        "disk_total_gb": round(disk.total / 1024**3, 1),
        "uptime_hours": uptime_hours,
    }
```

12.9 INTERFACES AND PROGRAM START

Each route returns the requested data as JSON. Most of them simply access the values that have already been collected. Only `api_history` does its own calculation: the per-minute values are averaged into larger time intervals in SQL, so that the chart for seven days does not consist of 10,000 points.

Finally, the four background loops are started as threads. The `daemon=True` option ensures that they are terminated automatically when the program exits.

```
@app.route("/")
def index():
    return render_template("index.html")


@app.route("/api/indoor")
```

```
def api_indoor():
    with sensor_lock:
        data = dict(sensor_data)
    data.update(indoor_extras(data["temperature"]))
    stufe, text, diff = pressure_trend(data["pressure"])
    data["pressure_trend"] = stufe
    data["pressure_trend_text"] = text
    data["pressure_change"] = diff
    return jsonify(data)


@app.route("/api/outdoor")
def api_outdoor():
    with weather_lock:
        return jsonify(weather_data)


@app.route("/api/forecast")
def api_forecast():
    with forecast_lock:
        return jsonify(forecast_data)


@app.route("/api/history")
def api_history():
    ranges = {"6h": (6 * 3600, 300), "24h": (86400, 900), "7d": (7 * 86400, 3600)}
    key = request.args.get("range", "24h")
    span, bucket = ranges.get(key, ranges["24h"])
    since = int(time.time()) - span
    with db_connect() as conn:
        rows = conn.execute(
            "SELECT (ts / ?) * ? AS b, AVG(in_temp), AVG(in_hum), AVG(out_temp) "
            "FROM readings WHERE ts >= ? GROUP BY b ORDER BY b",
            (bucket, bucket, since),
        ).fetchall()
    points = [
        [b, *(round(v, 1) if v is not None else None for v in vals)]
        for b, *vals in rows
    ]
    return jsonify({"range": key, "since": since, "points": points})


@app.route("/api/pi-stats")
def api_pi_stats():
    return jsonify(get_pi_stats())


@app.route("/api/location")
def api_location():
    return jsonify(
        {
            "lat": locations.current()["lat"],
            "lon": locations.current()["lon"],
            "name": locations.current()["name"],
            "night_start": NIGHT_START,
            "night_end": NIGHT_END,
            "idle_return": IDLE_RETURN_SECONDS,
        }
    )


@app.route("/api/map-key")
def api_map_key():
    # Wird nur im eigenen Netzwerk aufgerufen, damit die Karte die Wetterebenen laden kann.
    return jsonify({"appid": config.OPENWEATHER_API_KEY})


if __name__ == "__main__":
    init_db()
    init_leds()
```

```
    threading.Thread(target=sensor_loop, daemon=True).start()
    threading.Thread(target=weather_loop, daemon=True).start()
    threading.Thread(target=forecast_loop, daemon=True).start()
    threading.Thread(target=history_loop, daemon=True).start()
    app.run(host="0.0.0.0", port=5000, threaded=True)
```

12.10 RECURRING PATTERNS IN THE CODE

Finally, three things that run through the entire code and that we deliberately solved this way:

- **Errors are caught, not passed on.** Every loop is wrapped in a try-except block. If a query fails, for example because the internet is briefly unavailable, a message is logged and the query is retried on the next run. The program therefore does not exit.
- **Collecting and serving are separated.** The loops write to the stores, the interfaces only read from them. As a result, the website responds within milliseconds, even if OpenWeather takes several seconds.
- **Settings are never hard-coded.** Everything you might want to change is in config.py or saved in the settings tab. This means nothing in the program itself needs to be adjusted when, for example, the location or a threshold changes.